

Claude Certified Architect – Foundations

Unofficial community study guide – <https://github.com/dnacentra/claude-certified-architect>

Claude Certified Architect – Foundations

Anthropic's technical certification for solution architects building production applications with Claude (exam code **CCAR-F**). 60 multiple-choice and multiple-response, scenario-based questions in 120 minutes, proctored and closed-book. Passing score: 720/1000. Candidates answer questions from 4 of 6 randomly selected scenarios. No penalty for guessing.

Part of a four-credential family: [Claude Certified Associate – Foundations \(CCA0-F\)](#), **Architect – Foundations (CCAR-F)** – this guide, [Architect – Professional \(CCAR-P\)](#), and [Developer – Foundations \(CCDV-F\)](#). Delivered via Pearson VUE (OnVUE online or test center), registered through the Anthropic Partner Academy. Per Exam Guide v1.0 (effective July 2026): \$125 per attempt, certification valid 12 months, up to 4 attempts per rolling 12 months (14/30/90-day waits after attempts 1–3). Renewal is free when done on time – review what changed since you certified and pass a non-proctored assessment; let it lapse and the full exam fee applies again. Access remains gated to the Claude Partner Network (registration needs a partner-domain email); the earlier "\$99, first 5,000 partner employees free" beta terms are superseded. Delivery moved to Pearson VUE and digital badging to Credly on 2026-06-30. Partner-tier discounts apply at checkout – 50% for Select, Preferred, and Global Premier partners, 100% for Global Premier partners through 2026-12-31 – and candidates must be at least 18.

Target: Solution architect with 6+ months experience with Claude APIs, Agent SDK, Claude Code, and MCP.

Current Model Lineup (September 2026)

The exam is model-agnostic, but scenario questions and code samples assume a current lineup. This guide's samples use `claude-opus-5-5`, the docs' current default for most workloads; the few samples that rely on forced `tool_choice` use `claude-sonnet-5`, because both current flagships reject it.

Model	ID	Context	Max output	Price (in / out per MTok)	Notes
Claude Fable 5.1	<code>claude-fable-5-1</code>	1M	128k	\$10 / \$50 (cache reads \$0.25)	Most capable widely released model (2026-09-01) – for demanding reasoning and long-horizon agentic work, or when Opus 5.5 at higher effort still falls short; thinking always on; no forced <code>tool_choice</code> ; thinking blocks bound to model + history
Claude Opus 5.5	<code>claude-opus-5-5</code>	1M	128k	\$4 / \$20 (cache reads \$0.20)	Default for most workloads (2026-09-22): long-running agentic coding and knowledge work; thinking always on and can't be disabled ; default effort <code>medium</code> ; no forced <code>tool_choice</code> ; bound thinking blocks
Claude Sonnet 5	<code>claude-sonnet-5</code>	1M	128k	\$2 / \$10	Best speed/intelligence balance; still accepts forced <code>tool_choice</code> ; no mid-conversation system messages, no task budgets; \$2 / \$10 is now the permanent price (the scheduled rise to \$3 / \$15 was cancelled)
Claude Haiku 4.5	<code>claude-haiku-4-5</code>	200k	64k	\$1 / \$5	Fastest; only current model still using <code>budget_tokens</code> thinking; no <code>effort</code> ; nearest retirement floor in the lineup (2026-10-15)

Claude Mythos 5.1 (`claude-mythos-5-1`) shares Fable 5.1's specs and pricing and is invitation-only under Project Glasswing, for defensive cybersecurity work. It is the same underlying

model served without the dual-use safety measures Fable 5.1 carries; its safeguards depend on the access program, so still handle `refusal` there. **Legacy but still available:** Opus 5, Fable 5, Opus 4.8, Opus 4.7, Opus 4.6, Opus 4.5, Sonnet 4.6, Sonnet 4.5. Opus 4.1 retired on 2026-08-05. Anthropic publishes retirement floors: Fable 5.1 not before 2027-09-01, Opus 5.5 not before 2027-09-22, Opus 5 not before 2027-07-24, Sonnet 5 not before 2027-06-30, Haiku 4.5 not before 2026-10-15 – the only floor inside the next year, with no successor announced, so check the deprecations page before committing new work to it.

Model IDs from the 4.6 generation onward are **dateless but still pinned snapshots**, not evergreen pointers. Query `client.models.list()` / `.retrieve(id)` for live `max_input_tokens`, `max_tokens`, and `capabilities` rather than hardcoding a table like this one.

What Fable 5.1 and Opus 5.5 change for architects (Fable 5.1 introduced three breaking changes on 2026-09-01; Opus 5.5 inherited all three on 2026-09-22 – details in Domain 2 §2.3, Domain 4 §4.7, Domain 5 §5.7):

1. `tool_choice: "any"` and `{"type": "tool", ...}` return **400**. Use `auto` plus an explicit instruction, `strict: true`, or structured outputs.
2. Thinking blocks are readable only by some models. Fable 5.1 / Mythos 5.1 read everyone's; Opus 5.5 reads Opus 5 and earlier Opus / Sonnet / Haiku blocks but not Fable or Mythos ones; no older model reads Fable 5.1's or Opus 5.5's. A fallback or router switch to a model that can't read them silently drops them (unbilled) and that model re-plans.
3. Editing, reordering, or removing earlier turns invalidates every later thinking block. Harnesses must be **append-only**: freeze `system` and `tools`, move mid-session changes into `role: "system"` messages (tool definitions included, via inline tools), and trim context server-side (compaction on demand / context editing) rather than on the client.

Opus 5.5 adds three more of its own, none of which break Fable 5.1 code: **thinking can't be disabled** (`{"type": "disabled"}` and `budget_tokens` are a 400 at every effort – lower `effort` instead); the **default effort is medium**, one level below Opus 5, so a request that omits `effort` now runs cheaper and shallower than it did (set it explicitly and re-sweep); and on the Claude API and Google Cloud, computer use only through the `computer_toolset_20260801` toolset (`computer_20251124` returns 400). Two behaviour changes arrive with no error: the short text Claude writes between tool calls comes back as `thinking` blocks (empty at the default `display`), and a biology classifier plus a `reasoning_extraction` refusal category join the cybersecurity one.

Fable 5.1 also requires 30-day data retention (no zero-data-retention orgs unless authorized). Priority Tier is not supported on Fable 5.1, Opus 5.5, Opus 5, or Sonnet 5 – and new Priority Tier commitments are no longer sold, so plan capacity around rate limits and the Batch tier instead.

Docs (since 2026-07): Anthropic split its documentation. API docs live at `platform.claude.com/docs/en/*`, Claude Code docs at `code.claude.com/docs/en/*`; the old `docs.anthropic.com` URLs still redirect. The SDK was renamed from "Claude Code SDK" to the **Claude Agent SDK** (`pip install claude-agent-sdk`, `npm install @anthropic-ai/claude-agent-sdk`).

Program (Exam Guide v1.0, effective July 2026): four credentials (CCA0-F, CCAR-F, CCAR-P, CCDV-F), proctored via Pearson VUE, registered through the Anthropic Partner Academy. \$125 for CCAR-F, 12-month validity – superseding the launch-period "\$99 / first 5,000 partner employees free" terms.

Refresh history: [CHANGELOG.md](#).

Exam Domains

Domain	Weight	Deep Dive
Agentic Architecture & Orchestration	27%	domains/d1-agentic-architecture.md
Tool Design & MCP Integration	18%	domains/d2-tool-design-mcp.md
Claude Code Configuration & Workflows	20%	domains/d3-claude-code-config.md
Prompt Engineering & Structured Output	20%	domains/d4-prompt-engineering.md
Context Management & Reliability	15%	domains/d5-context-reliability.md

The 6 Exam Scenarios

1. **Customer Support Resolution Agent** – Agent SDK, MCP tools, escalation logic
2. **Code Generation with Claude Code** – CLAUDE.md, plan mode, slash commands
3. **Multi-Agent Research System** – Coordinator-subagent patterns, error handling
4. **Developer Productivity with Claude** – Built-in tools, MCP servers, codebase exploration
5. **Claude Code for CI/CD** – Structured output, batch API, multi-pass review
6. **Structured Data Extraction** – JSON schemas, tool_use, validation-retry

The 10 Critical Anti-Patterns

These show up as wrong answers on the exam:

1. Parsing natural language for loop termination instead of checking `stop_reason`
2. Arbitrary iteration caps as primary stopping mechanism
3. Prompt-based enforcement for critical business rules (should use hooks)
4. Self-reported confidence scores for escalation decisions
5. Sentiment-based escalation (sentiment $\neq$ complexity)
6. Generic error messages hiding diagnostic context
7. Silently suppressing errors (returning empty results as success)
8. Too many tools per agent (18 vs 4-5 recommended) – see Domain 2 §2.3 for how tool search changes this in the current API
9. Same-session self-review (retains reasoning context bias)
10. Aggregate accuracy metrics masking per-document-type failures

Key Decision Frameworks

Decision	Option A	Option B
Enforcement	Programmatic hooks – financial/safety	Prompt-based – best-effort
Execution mode	Plan mode – multi-file, architectural	Direct – single-file, obvious
tool_choice	"any" – guarantee tool call	Forced – specific tool first. <i>Both return 400 on Fable 5.1 / Mythos 5.1 / Opus 5.5</i> – there, <code>auto</code> + instruction + <code>strict: true</code> , or <code>output_config.format</code>
API type	Synchronous – blocking workflows	Batch – latency-tolerant
Error handling	Structured context – category, retry, partial	Never generic or silent
Escalation	Immediate – explicit customer request	Resolve first – within capability
Code review	Multi-pass – large PRs	Single-pass – small, focused
Context passing	Always explicit in subagent prompts	Never rely on auto-inheritance
Thinking	Adaptive (<code>{"type": "adaptive"}</code>) on every current model	<code>budget_tokens</code> is removed – 400 on Fable 5.x / Opus 5.5 / Opus 5 / Sonnet 5 / 4.7 / 4.8; <code>disabled</code> is also a 400 on Fable 5.x and Opus 5.5 (lower <code>effort</code> instead)
Effort	Start at the model's default and sweep – <code>high</code> on Fable 5.1 / Opus 5 / Sonnet 5, <code>medium</code> on Opus 5.5 (set it explicitly); <code>xhigh</code> for the hardest coding/agent work, and the recommended start on Opus 4.8 / 4.7	<code>low</code> / <code>medium</code> – subagents, mechanical and routine tasks
Tool exposure	Load upfront – under ~10 tools	Tool search + <code>defer_loading</code> – 10+ tools or multi-server MCP
Context reduction	Clear (context editing) – verbose tool results	Compact – long conversations that need their narrative; prefer on-demand compaction (you pick the moment, keep recent turns, run it in the background) over threshold compaction
State that must persist	Memory tool / files	Never rely on the conversation surviving compaction
Agent harness	Tool Runner / Agent SDK – you host	Managed Agents – Anthropic hosts loop + sandbox
Conversation history	Append-only – freeze <code>system</code> / <code>tools</code> , changes via <code>role: "system"</code> messages (inline <code>tool_addition</code> definitions for tools)	Never edit or snip earlier turns on Fable 5.1 / Opus 5.5 – it invalidates thinking blocks
Progress UX between tool calls	<code>thinking.display: "updates"</code> (beta) or <code>"summarized"</code> – Fable 5.x / Opus 5.5 return those notes as <code>thinking</code> blocks	Default <code>"omitted"</code> – the UI goes quiet between tool calls, with no error
Many-agent orchestration (Claude Code)	Subagents / forks – a few delegated tasks per turn	Dynamic workflows – a script the runtime executes for dozens to hundreds of agents

4-Week Study Plan

Assumes daily study, 1.5–2 hours per day.

Week	Focus	Topics	Daily Goal
1	Core Architecture	Agentic loops, <code>stop_reason</code> , multi-agent orchestration, hooks, session management, tool design, MCP config	1 domain section + practice questions
2	Applied Skills	CLAUDE.md hierarchy, plan mode, CI/CD, Batches API, explicit criteria, few-shot, validation-retry, structured output	1 domain section + build a small project using each concept
3	Reliability + Hands-On	Context management, escalation, error propagation, provenance, hands-on exercises across all domains	Review weak areas + build an end-to-end scenario
4	Exam Prep	Practice exams, anti-pattern drills, scenario walkthroughs, timed question sets	1 full practice exam per day + targeted review

Resources

- [Official CCAR-F Exam Guide \(PDF, v1.0\)](#) – the March 2026 launch guide is superseded
- [Anthropic Partner Academy – CCAR-F registration](#) · [Prep courses](#) · [Certification FAQ](#) (renewal, retakes, eligibility)
- [Pearson VUE – Anthropic certification program](#)
- [Anthropic Skilljar – Building with Claude API](#)
- [12-Week Training Program \(GitHub\)](#)
- [Building Effective Agents \(Anthropic Research\)](#)
- [Claude Partner Network](#)

API (platform.claude.com)

- [Tool use overview](#)
- [Tool search tool](#)
- [Programmatic tool calling](#)
- [Memory tool](#)
- [MCP connector](#)
- [Thinking](#) · [Preserved thinking](#) · [Effort](#) · [Task budgets](#)
- [Mid-conversation system messages](#) (incl. tool changes and defining tools in a message)
- [Context editing](#) · [Compaction overview](#) · [Compaction on demand](#) · [Keeping recent turns](#) · [Compaction and preserved thinking](#)
- [Prompt caching](#) · [Cache diagnostics](#)
- [Structured outputs](#)
- [Stop reasons and fallback](#) · [Refusals and fallback](#)
- [Managed Agents overview](#) · `ant` CLI · `ant apply` · [Admin API](#)
- [Models overview](#) · [What's new in Claude Opus 5.5](#) · [Opus 5.5 migration guide](#) · [Prompting Opus 5.5](#) · [Claude Fable 5.1 migration guide](#) · [Model deprecations](#) · [Pricing](#)

Claude Code (code.claude.com)

- [Overview](#) · [CLI reference](#)
- [Subagents](#) · [Dynamic workflows](#) · [Agent teams](#) · [Skills](#) · [Plugins](#) · [Hooks](#)
- [MCP](#) · [Memory and CLAUDE.md](#) · [Routines](#) · [Commands](#) · [Changelog](#)
- [Claude Agent SDK overview](#)

Background reading

- [MCP Introduction](#)
- [Advanced tool use \(Anthropic Engineering\)](#)
- [Effective context engineering for AI agents](#)

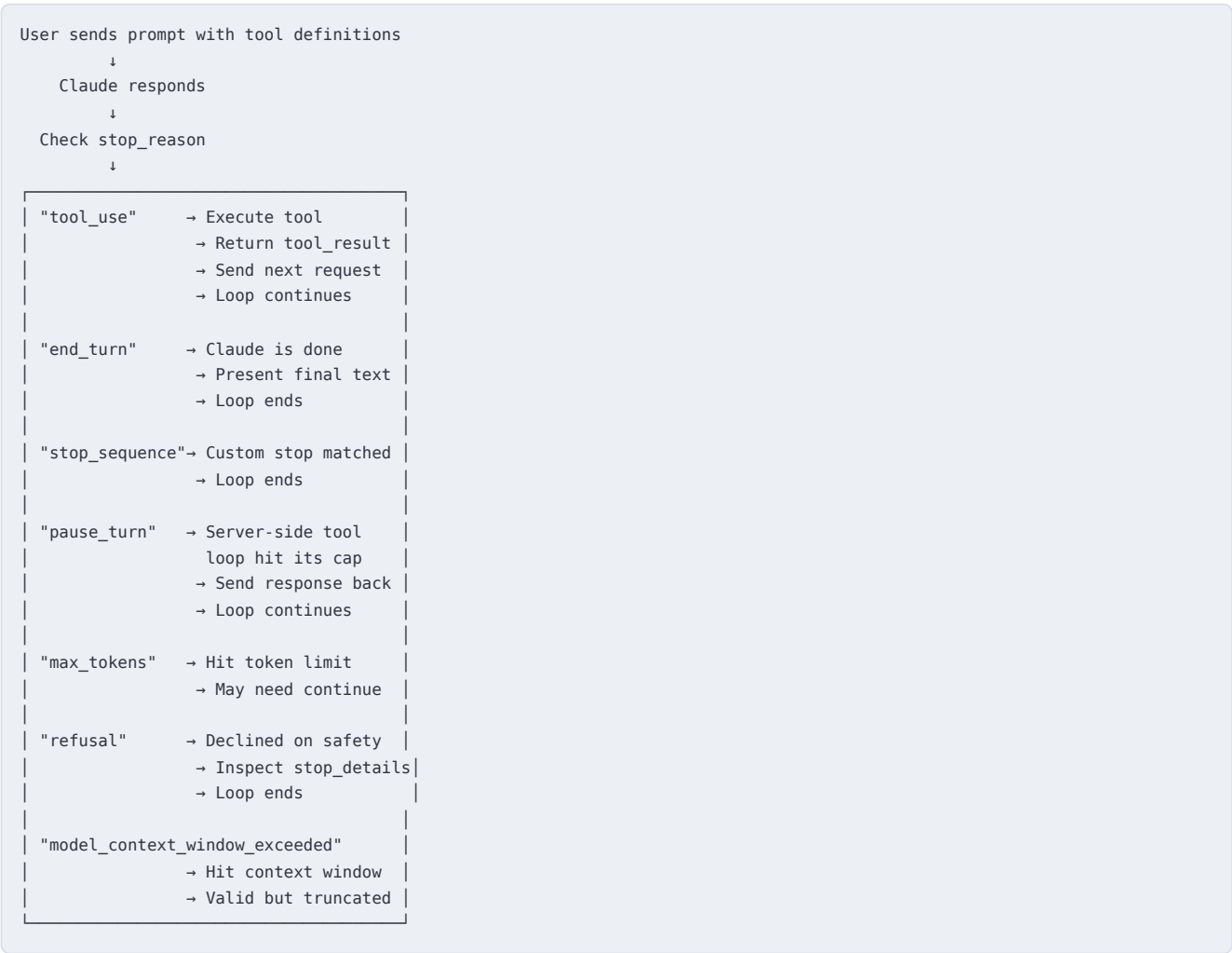
Domain 1: Agentic Architecture & Orchestration (27%)

The heaviest domain on the exam. Covers 7 task statements focused on how agentic systems are built, how loops operate, how agents coordinate, and how enforcement works.

1.1 The Agentic Loop Lifecycle

At its core, an agentic system is a loop. Claude receives a prompt with tools, decides whether to call a tool or respond, and the loop continues until Claude signals it's done.

How the Loop Works



The stop_reason Values

Value	Meaning	Action
"tool_use"	Claude wants to call a tool	Execute it, return result, continue loop
"end_turn"	Claude has finished reasoning	Present the final text response
"stop_sequence"	Output matched a configured stop_sequences string	Loop ends; inspect which sequence was hit
"pause_turn"	Server-side tool loop (e.g., web_search) hit its internal iteration limit	Send the response back to continue
"max_tokens"	Response hit the max_tokens limit	May need to request continuation
"refusal"	Claude declined to respond on safety grounds	Loop ends; inspect stop_details (see below), rephrase or route the request
"model_context_window_exceeded"	Generation hit the model's context window before max_tokens	Response is valid but truncated; trim input or continue. Default in Sonnet 4.5+

stop_details on refusals (Opus 4.7+): A refusal response also carries a stop_details object (no beta header needed). stop_details.type is always "refusal"; stop_details.category is the policy category ("cyber", "bio", "frontier_llm", "reasoning_extraction", "general_harms", or null – the set grows over time); stop_details.explanation is a human-readable string (don't parse it). stop_details is null for every other stop reason. Use the category to route or log specific refusals differently. Since 2026-09-24 a refusal that arrives before any output is billed when its category is bio, frontier_llm, or reasoning_extraction (the low-false-positive categories); cyber, general_harms, and null refusals before output are still free. Mid-stream refusals were always billed, and every refusal counts against rate limits.

Server-Side Refusal Fallbacks

A refusal arrives as HTTP 200, not an exception – code that reads content without checking stop_reason first will silently process an empty or partial response. For production agents on Claude Opus 5.5, Opus 5, and Fable 5 / 5.1 (all four ship safety classifiers; Mythos 5.1's safeguards depend on its access program, so handle refusal there too), don't just log the refusal: opt into server-side fallback so the request is re-routed automatically by refusal category.

```
response = client.messages.create(
    model="claude-opus-5-5",
    max_tokens=16000,
    betas=["server-side-fallback-2026-07-01"],
    fallbacks="default",           # server routes by refusal category
    messages=[...],
)
```

fallbacks="default" means you never maintain a model list. The older array form (betas=["server-side-fallback-2026-06-01"] + fallbacks=[{"model": "claude-opus-4-8"}]) still works. Server-side fallback is Claude API only – on Bedrock, Vertex, and Foundry use the SDKs' client-side BetaRefusalFallbackMiddleware instead.

One consequence on Fable 5.1 and Opus 5.5: a fallback usually lands on a model that cannot read their thinking blocks. Fable 5.1 / Mythos 5.1 read everything (each other's, Opus 5.5's, and earlier models'); Opus 5.5 reads Opus 5 and earlier Opus / Sonnet / Haiku blocks but not Fable or Mythos ones; no older model reads Fable 5.1's or Opus 5.5's. The API drops unreadable blocks (unbilled), the request succeeds, and the fallback model re-plans without that reasoning – expect a slower, costlier first turn after the switch. Send the `thinking-binding-controls-2026-08-01` beta header to get an `input_transformations` list of what was dropped.

Loop rule: always branch on `stop_reason` before reading `content`. `refusal` is a terminal stop reason for that request, not a retryable tool error. And on Fable 5.1 and Opus 5.5, keep the `messages` array **append-only** – editing or snipping earlier turns invalidates every later thinking block (Domain 5 §5.7).

API Response Structure

When Claude wants to use a tool, the response contains both text and a `tool_use` block:

```
{
  "id": "msg_01Aq9w938a90dw8q",
  "model": "claude-opus-5-5",
  "stop_reason": "tool_use",
  "role": "assistant",
  "content": [
    {
      "type": "text",
      "text": "I'll check the current weather in San Francisco for you."
    },
    {
      "type": "tool_use",
      "id": "toolu_01A09q90qw90lq917835lq9",
      "name": "get_weather",
      "input": { "location": "San Francisco, CA", "unit": "celsius" }
    }
  ]
}
```

The `content` array can contain multiple blocks – Claude may explain what it's doing (text) and call a tool (`tool_use`) in the same response. It can also call multiple tools simultaneously by including multiple `tool_use` blocks.

Returning Tool Results

Tool results must reference the exact `tool_use_id` from Claude's request:

```
{
  "role": "user",
  "content": [
    {
      "type": "tool_result",
      "tool_use_id": "toolu_01A09q90qw90lq917835lq9",
      "content": "15 degrees celsius, partly cloudy"
    }
  ]
}
```

For multiple simultaneous tool calls, return multiple `tool_result` blocks in the same message, each matching its `tool_use_id`.

Error Results

When a tool execution fails:

```
{
  "type": "tool_result",
  "tool_use_id": "toolu_01A09q90qw90lq917835lq9",
  "is_error": true,
  "content": "API timeout after 30 seconds"
}
```

Setting `is_error: true` helps Claude understand the tool failed and reason about recovery strategies.

Model-Driven Decision Making

The exam tests that you understand: **Claude decides** which tool to call and when to stop. You don't pre-configure decision trees or fixed tool sequences. The model reasons about what information it needs, selects the appropriate tool, processes the result, and decides the next step.

This is fundamentally different from traditional workflow automation where steps are predetermined.

Anti-Patterns (These Are Wrong Answers)

1. **Parsing natural language** for loop termination – "If Claude says 'I'm done', exit the loop." Wrong. Always check `stop_reason`.
2. **Arbitrary iteration caps** as primary stopping – "Run max 5 loops then stop." Wrong as a primary mechanism. You may use safety caps, but `stop_reason` drives the loop.
3. **Checking for text content** as completion – "If the response has text, it's done." Wrong. Responses can contain both text and tool_use blocks.

Beyond the exam guide – task budgets (beta): the modern answer to "how do I bound an agentic loop without a hard iteration cap" is a **task budget**. `output_config.task_budget` gives Claude a token ceiling it can see, so it paces itself and finishes gracefully instead of being cut off mid-work. This is different from `max_tokens`, which is an enforced per-response ceiling the model is unaware of.

```
with client.beta.messages.stream(
    model="claude-opus-5-5", max_tokens=128000,
    betas=["task-budgets-2026-03-13"],
    output_config={"effort": "high",
                  "task_budget": {"type": "tokens", "total": 64000}},
    messages=[...], tools=[...],
) as stream:
    response = stream.get_final_message()
```

Minimum `total` is 20,000. Available on Claude Fable 5.1 / Mythos 5.1, Fable 5 / Mythos 5, Opus 5.5, Opus 5, and Opus 4.8/4.7 – **not** on Sonnet 5, Opus 4.6, or Haiku 4.5, and not through Claude Code or Cowork (Messages API only). Stream it – a large `max_tokens` on a non-streaming request hits HTTP timeouts. Size the budget against your real task-length distribution: a budget that is obviously too small makes Claude decline, scope down, or stop early with a partial result, which looks like a refusal but isn't one. The exam's anti-pattern still holds: a budget paces the loop, `stop_reason` still terminates it.

Who Runs the Loop: Four Ways to Build an Agent

Two independent questions separate the options: **who supplies the harness** (the loop plus context management) and **who supplies the deployment** (the infrastructure it runs on). Tool

Runner and the Claude Agent SDK are easy to conflate because both supply a harness only – you still host them.

Approach	You write	Harness & deployment	Tools available
Manual loop (<code>client.messages.create</code>)	The <code>while</code> <code>stop_reason</code> <code>==</code> <code>"tool_use"</code> loop	You build the harness; you host	Only tools you define
Tool Runner (<code>client.beta.messages.tool_runner</code>)	Just the tool functions	SDK supplies the loop; you host	Only tools you define
Managed Agents (beta, <code>/v1/agents</code> + <code>/v1/sessions</code>)	Agent config + your tool results	Anthropic supplies the loop and hosts a per-session sandbox	Hosted bash / files / code execution, plus Skills, MCP, and your tools
Claude Agent SDK (<code>claude-agent-sdk</code>)	A prompt + options	Claude Code harness + built-in tools; you host	Read/Write/Edit/Bash/Glob/Grep/WebSearch/WebFetch + MCP + subagents

Tool Runner ≠ Claude Agent SDK. Tool Runner ships inside the regular Anthropic SDK (`anthropic` / `@anthropic-ai/sdk`) and only loops over tools *you* define – no built-in tools, no filesystem, no sandbox. It exposes per-turn hooks for approval gates, error interception, result modification (e.g. attaching `cache_control`), and retries. The Claude Agent SDK is Claude Code packaged as a library.

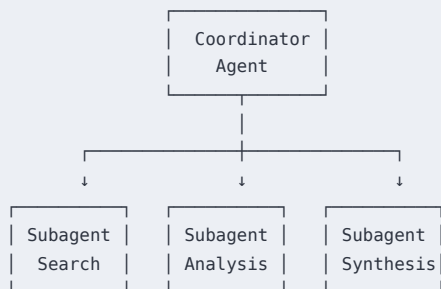
Managed Agents (beta) is the newest surface and the only one that adds managed *deployment*. The mandatory flow is Agent (created once, versioned, persisted) → Session (one per run). `model`, `system`, and `tools` live on the **agent**, never the session. Each session provisions a container that acts as the agent's workspace and streams events back; you send messages and tool results in. It also adds scheduled deployments (cron-fired sessions), vault-stored credentials substituted at egress, dollar-denominated session budgets, and multiagent rosters. Beta header: `managed-agents-2026-04-01`. Not available on Bedrock / Vertex / Foundry – use Claude API + tool use there. The recommended control-plane flow is now the `ant` **CLI**: describe agents, environments, skills, memory stores, and deployments as files in your repository and run `ant apply` (v1.30.0, 2026-09-03), which prints a plan, applies it, and writes a `claude-lock.json` lockfile – commit it so later runs, locally or in CI, update the same resources instead of creating new ones (`ant beta:agents create < agent.yaml` still works for one-offs). Application code owns only the data plane (`sessions.create` with the stored agent ID). `ant auth login` also gives the SDKs an OAuth profile, so a bare `Anthropic()` client works with no API key in the environment. Two September additions: permission policies gained `auto` – the server evaluates each agent or MCP tool call and runs it, denies it as high-risk, or pauses for your approval, reporting the verdict in an `evaluation` field on `agent.tool_use` events – and `ant beta:sessions connect` attaches your terminal to a live session to follow it, send messages, and approve or deny waiting tool calls (`--web` serves the Console's session viewer locally).

Choosing: stay at the simplest tier that works. A single call or a code-controlled workflow handles most tasks; reach for an agent only when the task is genuinely open-ended and model-driven, the value justifies the latency and cost, and errors are catchable (tests, review, rollback).

1.2 Multi-Agent Orchestration

Hub-and-Spoke Architecture

The standard pattern for multi-agent systems is hub-and-spoke: one coordinator agent manages multiple specialized subagents.



Coordinator Responsibilities

The coordinator:

- **Decomposes** the task into subtasks
- **Delegates** to appropriate subagents
- **Routes** all inter-subagent communication (subagents never talk to each other directly)
- **Aggregates** results from subagents
- **Handles errors** from subagent failures
- **Evaluates** synthesis quality and re-delegates if gaps exist

Subagent Isolation

Critical concept: Subagents have isolated context. They do NOT inherit the coordinator's conversation history. Everything a subagent needs must be explicitly passed in its prompt.

This means:

- The coordinator must include all relevant context in the subagent's task description
- Subagents can't reference earlier parts of the main conversation
- Each subagent starts fresh with only what the coordinator provides

Dynamic Subagent Selection

Not every query needs every subagent. The coordinator should:

- Assess what the query actually requires
- Only invoke relevant subagents
- Not route everything through a full pipeline

Example: A research query about a single topic doesn't need web search + document analysis + synthesis if a single focused search would suffice.

Decomposition Risks

Overly narrow decomposition: The exam specifically tests this. Example scenario: coordinator is asked to research "creative industries" and decomposes it into only visual arts subtasks, missing music, writing, film, and design.

The fix: broader initial decomposition with explicit coverage checks.

Iterative Refinement Loops

The coordinator evaluates synthesis output and identifies gaps:

```
Coordinator → Subagents (round 1)
↓
Evaluate results
↓
Identify gaps
↓
Coordinator → Subagents (round 2, targeted)
↓
Final synthesis
```

1.3 Subagent Spawning and Context Passing

The Agent Tool

Subagents are spawned with the `Agent` tool – the canonical name in both Claude Code and the Claude Agent SDK. The coordinator must have `Agent` in its allowed tools to spawn subagents, and access can be narrowed to specific subagent types with `Agent(worker, researcher)`.

Naming note: older material (including the exam guide's wording) calls this the **Task** tool. `Task` was Claude Code's original name for the same mechanism; current Claude Code and SDK docs use `Agent`. If an exam item says "Task tool", it means this.

SDK naming: the SDK was renamed from "Claude Code SDK" to the **Claude Agent SDK**. Python: `pip install claude-agent-sdk`. TypeScript: `npm install @anthropic-ai/claude-agent-sdk`. Top-level entry point: `query()` + `ClaudeAgentOptions`.

Context Must Be Explicit

This is tested heavily: **context does not automatically inherit**. When the coordinator spawns a subagent, it must include everything the subagent needs in the prompt.

Bad:

```
"Analyze the customer's issue"
# Subagent has no idea who the customer is or what the issue is
```

Good:

```
"Customer #12345 reported that their order #67890 (placed 2024-01-15,
total $149.99) arrived damaged. They are requesting a full refund.
Analyze the order history and return policy applicability."
```

Parallel Subagent Execution

To run subagents in parallel, emit multiple `Agent` tool calls in a **single coordinator response**. Not across separate turns – that would be sequential.

```
{
  "content": [
    {
      "type": "tool_use",
      "name": "Agent",
      "input": { "prompt": "Search for recent papers on...", "subagent_type": "search" }
    },
    {
      "type": "tool_use",
      "name": "Agent",
      "input": { "prompt": "Analyze the document at...", "subagent_type": "analysis" }
    }
  ]
}
```

Claude Code caps concurrent subagents at 20 (`CLAUDE_CODE_MAX_CONCURRENT_SUBAGENTS`) and spawn depth at 3 (`CLAUDE_CODE_MAX_SUBAGENT_SPAWN_DEPTH`); at the depth limit the `Agent` tool is withheld from the subagent (a fork keeps it listed, but it errors instead of spawning).

Beyond the exam guide – when the fan-out outgrows a turn: Claude Code now has four orchestration units, and the exam's hub-and-spoke reasoning maps onto the first one. The difference is *who holds the plan*.

	Subagents	Skills	Agent teams (experimental)	Dynamic workflows
What it is	A worker Claude spawns	Instructions Claude follows	A lead session supervising peer sessions	A JavaScript script the runtime executes
Who decides what runs next	Claude, turn by turn	Claude, following the prompt	The lead agent	The script (<code>agent()</code> , <code>parallel()</code> , <code>pipeline()</code>)
Intermediate results live in	Claude's context	Claude's context	A shared task list	Script variables
Scale	A few delegated tasks per turn	Same	A handful of long-running peers	Dozens to hundreds of agents per run, resumable

A workflow moves the coordinator's plan out of the model and into code – the deterministic answer to "the coordinator forgot half the subtasks" (§1.2). Claude writes the script; `/workflows` watches it; `ultracode` (an effort setting) lets Claude decide when to reach for one. Agent teams need `CLAUDE_CODE_EXPERIMENTAL_AGENT_TEAMS=1`. The medium workflow-size guideline is now 10 agents (was 15), Pro plans default to small, and `CLAUDE_CODE_WORKFLOW_MAX_CONCURRENT_AGENTS` (1–256) raises a run's concurrency cap for inference-bound fan-outs.

AgentDefinition in the Claude Agent SDK

```
from claude_agent_sdk import query, ClaudeAgentOptions, AgentDefinition

async for message in query(
    prompt="Use the code-reviewer agent to review this codebase",
    options=ClaudeAgentOptions(
        allowed_tools=["Read", "Glob", "Grep", "Agent"],
        agents={
            "code-reviewer": AgentDefinition(
                description="Expert code reviewer for Python and TypeScript",
                prompt="Analyze code quality, identify bugs, suggest improvements.",
                tools=["Read", "Glob", "Grep"],
            ),
            "test-writer": AgentDefinition(
                description="Test generation specialist",
                prompt="Write comprehensive unit tests for the provided code.",
                tools=["Read", "Write", "Bash"],
            ),
        },
    ),
):
    print(message)
```

Messages emitted from inside a subagent's context carry a `parent_tool_use_id` field – use it to attribute messages to the right subagent execution.

Forks vs Fresh Subagents

A **fork** is the opposite trade-off from an isolated subagent: it inherits the *entire* conversation – history, system prompt, tools, model – and shares the prompt cache, so it is cheaper than a fresh subagent. Only its final result returns to the main conversation; its tool calls stay isolated. Forks can't spawn sub-forks.

	Fresh subagent	Fork
Conversation history	Not inherited – pass everything explicitly	Fully inherited
System prompt / tools	The subagent's own	Same as the parent session
Prompt cache	Cold	Shared with the parent (cheaper)
Use for	Bounded, well-specified tasks; noisy exploration	Divergent branches off a shared baseline

In Claude Code, Claude requests a fork through the `Agent` tool with `subagent_type: "fork"` (fork mode is on by default in interactive sessions, and subagents it spawns then run in the background). You can start one yourself with `/subtask <prompt>`. Two neighbours are easy to confuse: `/fork` now *copies* the whole session into a separate background session (agent view), and `/branch` switches you into a copy of the conversation to try a different direction. The SDK/CLI also exposes `--fork-session` for resuming a session under a new ID; `fork_session` is the older API-level name for the same "branch from a shared baseline" idea.

Structured Context Passing

Best practice: separate content from metadata when passing context between agents.

```
{
  "findings": [
    {
      "claim": "Revenue increased 15% YoY",
      "evidence": "Q4 earnings report, page 3",
      "source_url": "https://example.com/earnings",
      "document_name": "Q4 2024 Earnings Report",
      "publication_date": "2025-01-15"
    }
  ]
}
```

This preserves attribution and allows the receiving agent to evaluate source quality.

1.4 Workflow Enforcement and Handoff

The Critical Distinction

This is one of the most heavily tested concepts on the exam:

Enforcement Type	When to Use	Guarantee Level
Programmatic (hooks, prerequisites)	Financial/safety consequences	Deterministic – 100% enforced
Prompt-based (instructions)	Best-effort acceptable	Probabilistic – non-zero failure rate

Example: Financial Operations

Scenario: A customer support agent must verify identity before processing refunds.

Wrong approach (prompt-based):

"Always verify the customer's identity before processing any refund."

This will work most of the time, but has a non-zero failure rate. For financial operations, "most of the time" is not acceptable.

Correct approach (programmatic): A PreToolUse hook on `process_refund` that checks whether `get_customer` has been called and returned a verified customer ID. If not, the hook blocks the refund with exit code 2.

Handoff to Humans

When an agent escalates to a human, the human agent typically does NOT have access to the full conversation transcript. The handoff must include a structured summary:

Customer: Jane Doe (#12345)
Issue: Damaged product received
Order: #67890, \$149.99, placed 2024-01-15
Root cause: Shipping damage
Actions taken: Verified order, confirmed damage via photos
Recommended action: Full refund + replacement shipment
Refund amount: \$149.99

1.5 Claude Code / Agent SDK Hooks

Hook Events (Current Catalog)

The hook system expanded substantially in 2026 – the current catalog is **33 events** across eight groups. Exam-relevant ones are marked *****.

Group	Events
Setup	<code>Setup</code> (runs once per session)
Session & turn	<code>SessionStart</code> *, <code>SessionEnd</code> *, <code>UserPromptSubmit</code> *, <code>UserPromptExpansion</code> , <code>Stop</code> *, <code>StopFailure</code>
Tool / agentic loop	<code>PreToolUse</code> *, <code>PostToolUse</code> *, <code>PostToolUseFailure</code> , <code>PostToolBatch</code> , <code>PermissionRequest</code> *, <code>PermissionDenied</code>
Agent & task	<code>SubagentStart</code> , <code>SubagentStop</code> *, <code>TaskCreated</code> , <code>TaskCompleted</code> , <code>TeammateIdle</code>
File & config	<code>FileChanged</code> , <code>CwdChanged</code> , <code>DirectoryAdded</code> , <code>ConfigChange</code> , <code>InstructionsLoaded</code>
Compaction	<code>PreCompact</code> *, <code>PostCompact</code>
Model	<code>PreModelSwitch</code> , <code>PostModelSwitch</code> (block, confirm, or annotate a model change – new in v2.1.251)
Context & worktree	<code>Notification</code> , <code>MessageDisplay</code> , <code>Elicitation</code> , <code>ElicitationResult</code> , <code>WorktreeCreate</code> , <code>WorktreeRemove</code>

The rough lifecycle you should carry into the exam:

```
SessionStart
  → UserPromptSubmit
  → [Agentic Loop]:
    PreToolUse
      → PermissionRequest (may fire PermissionDenied)
      → PostToolUse / PostToolUseFailure
      → SubagentStart / SubagentStop
      → TaskCreated / TaskCompleted
    → Stop / StopFailure
  → PreCompact → PostCompact
  → PreModelSwitch → PostModelSwitch (only when the model changes)
  → SessionEnd
```

Hook Types

Five types are supported. `mcp_tool` is new in 2026.

Type	How It Works	Use Case
<code>command</code>	Runs a shell command. Exit 0 = pass, exit 2 = block	File validation, linting gates
<code>http</code>	POST to a URL endpoint	External audit logging, webhooks
<code>mcp_tool</code>	Calls a tool on a configured MCP server	Structured validation via a reusable service
<code>prompt</code>	Single LLM call that returns a yes/no decision	Context-dependent approval
<code>agent</code>	Multi-turn subagent with tool access	Complex compliance checks

Two constraints from recent releases: `PermissionRequest` no longer runs `agent`-type hooks – their answer could never allow or deny the request, so use `command` or `http` there (v2.1.280+) – and `mcp_tool` hooks on blocking events wait for their MCP server to finish connecting instead of being skipped.

PreToolUse Hooks (Current JSON Shape)

Each matcher holds an **array** of `hooks` (the old singular `hook` field is gone). Hooks can be `async`, gated on `if` permission rules, and carry a `statusMessage` shown in the UI.

```
{
  "hooks": {
    "PreToolUse": [
      {
        "matcher": "process_refund",
        "hooks": [
          {
            "type": "command",
            "if": "Bash(git *)",
            "command": "\\\"$CLAUDE_PROJECT_DIR\\\"/.claude/hooks/check-refund-authorization.sh",
            "timeout": 600,
            "statusMessage": "Verifying refund authorization...",
            "shell": "bash"
          },
          {
            "type": "mcp_tool",
            "server": "compliance",
            "tool": "check_refund_policy",
            "input": { "amount": "${tool_input.amount}" },
            "timeout": 60
          }
        ]
      }
    ]
  },
  "disableAllHooks": false
}
```

Decision control output (unchanged):

```
{
  "hookSpecificOutput": {
    "hookEventName": "PreToolUse",
    "permissionDecision": "deny",
    "permissionDecisionReason": "Customer identity not yet verified",
    "additionalContext": "Call get_customer first to verify identity"
  }
}
```

`permissionDecision` values: `"allow"`, `"deny"`, `"ask"`.

PostToolUse Hooks

Intercept tool results before Claude processes them. Useful for:

- Normalizing timestamps, date formats, status codes
- Scrubbing sensitive data from results
- Adding metadata to results

If the tool itself errored, `PostToolUseFailure` fires instead – handle retries and diagnostics there.

Hooks in Skills and Subagents

Hooks can now be scoped to a **skill's lifecycle** via the `hooks:` frontmatter field in `SKILL.md`, and subagents can carry their own hook definitions. This lets you enforce rules locally (e.g., a `commit` skill enforces `cargo fmt` before `git commit`) without polluting project-wide settings.

Hook Configuration Locations

Location	Scope	Shared?
<code>~/.claude/settings.json</code>	All projects for this user	No (personal)
<code>.claude/settings.json</code>	This project	Yes (version control)
<code>.claude/settings.local.json</code>	This project, this machine	No (gitignored)
Managed policy settings	Organization-wide	Yes (admin-managed)

1.6 Task Decomposition Strategies

Prompt Chaining (Fixed Sequential)

For predictable, multi-aspect tasks where you know the steps upfront:

```
Step 1: Analyze each file individually (local analysis)
Step 2: Cross-file integration pass (data flow, consistency)
Step 3: Generate final report
```

Each step's output feeds into the next. The sequence is predetermined.

Dynamic Adaptive Decomposition

For open-ended investigation where subtasks emerge from discoveries:

```
Initial query: "Why is the app slow?"
→ Profile CPU usage
  → Discovery: database queries are the bottleneck
    → Analyze query patterns
      → Discovery: N+1 queries in the user listing endpoint
        → Generate fix for specific queries
```

The subtasks are generated based on what each step reveals.

Multi-Pass Review Architecture

For code review of large PRs:

Pass 1 – Per-file local analysis:

- Each file reviewed independently
- Consistent depth across all files
- Catches local issues (bugs, style, types)

Pass 2 – Cross-file integration:

- Separate pass looking at how files interact
- Catches data flow issues, interface mismatches, cross-file consistency
- Different prompt focused on integration concerns

Why two passes? Single-pass review of large PRs leads to:

- Attention dilution (model focuses on early files, skimps on later ones)
- Contradictory findings (inconsistent standards across files)
- Missed integration issues (can't see cross-file problems when reviewing one file)

1.7 Session Management

Resuming Sessions

`--resume <session-name>` continues a named prior conversation with full context.

When to resume:

- Ongoing investigation where tool results are still valid
- Iterative development within the same session

When to start fresh:

- Tool results are stale (files changed, services restarted)
- Context has drifted too far from current task
- In these cases, pass a structured summary of prior findings rather than resuming

Forking a Session

`--fork-session` resumes an existing session under a **new** session ID, so the original stays untouched and each branch explores a different approach without contaminating the others. In an interactive session, `/subtask <prompt>` spawns a fork that inherits the full conversation and returns only its result, `/fork` copies the session into a separate background session, and `/branch` switches you into a copy (see §1.3).

Use case: Testing two different refactoring strategies from the same starting point.

Moving a Session Between Surfaces

Sessions are no longer tied to one machine. `claude --cloud "<task>"` starts the work in a cloud session (`--remote` is a deprecated alias); `claude --teleport` pulls a web session back into the terminal; `/desktop` hands the current terminal session to the desktop app for visual diff review. Remote Control drives a running local session from a phone or another browser. Locally, `claude --bg "<task>"` starts a **background session** and returns immediately; `claude agents` opens the agent view, and `claude attach / logs / stop / respawn / rm <id>` manage it from the shell (`respawn --all` restarts every running session, e.g. to pick up a new binary). This matters architecturally: a "session" is a portable unit of state, so anything the agent must not lose belongs in files or memory, not in scrollbar.

Informing Resumed Sessions

When resuming a session after file changes, you must explicitly tell Claude what changed:

"Since our last session, I've updated auth.py to use JWT tokens instead of session cookies. Please re-analyze the authentication flow with these changes."

Don't assume the resumed session will notice changes on its own – tool results from the previous session may reflect old file contents.

Domain 1 Practice Questions

Q1: An agentic loop is processing customer requests. After Claude responds, what should the system check to determine the next action?

- A) Whether the response contains text content
- B) The `stop_reason` field in the response
- C) The length of the response

- D) Whether Claude expressed confidence in its answer

Answer: B – The `stop_reason` field determines whether to continue the loop (`tool_use`), end it (`end_turn`), or handle edge cases (`pause_turn`, `max_tokens`).

Q2: A coordinator agent needs to pass customer information to a subagent. What is the correct approach?

- A) The subagent will automatically inherit the coordinator's conversation context
- B) Include all relevant customer information explicitly in the subagent's prompt
- C) Store the information in a shared database that both agents access
- D) Use environment variables to pass the context

Answer: B – Subagents have isolated context and do not inherit the coordinator's conversation. Context must be explicitly passed in the prompt.

Q3: A financial services application needs to ensure identity verification occurs before any refund processing. Which enforcement mechanism should be used?

- A) Add instructions to the system prompt requiring verification first
- B) Use a PreToolUse hook that blocks `process_refund` until verification is complete
- C) Train the model on examples where verification comes first
- D) Set a high temperature to encourage creative verification approaches

Answer: B – Financial operations require deterministic, programmatic enforcement. Prompt-based instructions have a non-zero failure rate and are inappropriate for financial/safety-critical workflows.

Domain 2: Tool Design & MCP Integration (18%)

Covers how to design tool interfaces that LLMs can reliably use, how to handle errors from tools, how to distribute tools across agents, and how MCP servers are configured and operate.

2.1 Tool Interface Design

Tool Descriptions Are Everything

Tool descriptions are the **primary mechanism** Claude uses to decide which tool to call. The description is not just documentation – it's the selection criteria.

Bad description (leads to unreliable selection):

```
{
  "name": "search",
  "description": "Search for things"
}
```

Good description:

```
{
  "name": "search_knowledge_base",
  "description": "Search the internal knowledge base for support articles, FAQs, and troubleshooting guides. Use this when you need help with common issues."
}
```

What to Include in Tool Descriptions

1. **Purpose** – What the tool does and when to use it
2. **Input format** – What the input should look like, with examples
3. **Output format** – What the tool returns
4. **Edge cases** – What happens with invalid inputs or no results
5. **Differentiation** – When to use THIS tool vs similar tools

The Full Tool Definition Schema

```
{
  "name": "get_weather",
  "description": "Get the current weather in a given location. Returns temperature, conditions, humidity, and wind speed.",
  "input_schema": {
    "type": "object",
    "properties": {
      "location": {
        "type": "string",
        "description": "The city and state/country, e.g. 'San Francisco, CA' or 'London, UK'"
      },
      "unit": {
        "type": "string",
        "enum": ["celsius", "fahrenheit"],
        "description": "Temperature unit. Defaults to celsius if not specified."
      }
    },
    "required": ["location"]
  }
}
```

Strict Mode

Strict mode guarantees Claude's tool inputs conform exactly to the schema:

```
{
  "name": "extract_invoice",
  "strict": true,
  "input_schema": {
    "type": "object",
    "properties": {
      "vendor_name": { "type": "string" },
      "invoice_number": { "type": "string" },
      "total_amount": { "type": "number" },
      "currency": { "type": "string", "enum": ["USD", "EUR", "GBP"] }
    },
    "required": ["vendor_name", "invoice_number", "total_amount", "currency"],
    "additionalProperties": false
  }
}
```

Requirements for strict mode:

- `additionalProperties: false` must be set
- All properties must be in `required`
- Only a subset of JSON Schema is supported (see Domain 4)

Anti-Pattern: Ambiguous Tool Overlap

Having two tools with near-identical descriptions causes misrouting:

```
Tool 1: "analyze_content" – "Analyze content for insights"
Tool 2: "analyze_document" – "Analyze a document for insights"
```

Claude can't reliably distinguish between these. Fix: rename to clearly differentiate purpose, or merge into one tool.

2.2 Structured Error Responses

MCP Error Structure

When tools fail, the error response should be structured, not a generic string:

```
{
  "isError": true,
  "errorCategory": "transient",
  "isRetryable": true,
  "description": "Database connection timed out after 30 seconds. The query was for customer #12345's order history.",
  "partial_results": null,
  "attempted_action": "SELECT * FROM orders WHERE customer_id = 12345"
}
```

Error Categories

Category	Retryable?	Examples	Agent Action
Transient	Yes	Timeouts, service unavailability, rate limits	Retry with backoff
Validation	No (without input changes)	Invalid email format, missing required field	Fix input, retry
Business	No	Refund exceeds policy limit, account frozen	Escalate or inform user
Permission	No	Insufficient access, expired token	Escalate

What the Agent Should Do with Errors

1. **Transient errors:** Subagents should implement local recovery (retry with backoff). Only propagate if recovery fails.
2. **Validation errors:** Claude should examine the error, fix the input, and retry.
3. **Business errors:** Claude should explain the policy to the user or escalate.
4. **Permission errors:** Claude should escalate to a human.

Anti-Patterns (Wrong Answers)

1. **Generic error messages:** `"Operation failed"` – hides the cause and prevents intelligent recovery
2. **Silent suppression:** Returning `{ "results": [] }` when the search engine is down. This makes it look like there are no results when actually the search didn't work. The agent will confidently tell the user "I found nothing" when the truth is "I couldn't search."
3. **Generic status messages:** `"search unavailable"` without context about what was attempted
4. **Terminating on single failure:** Killing an entire multi-step workflow because one step failed, instead of using partial results or trying alternatives

Distinguishing Empty from Failed

This is a specific exam concept:

Situation	What Happened	Correct Response
Search returns <code>[]</code> with status 200	Valid query, no matches	"No results found for that query"
Search returns timeout error	Query didn't execute	"I wasn't able to search right now, let me try again"

These must be handled differently. The first is information. The second is an error.

2.3 Tool Distribution and tool_choice

The 4-5 Tool Maximum

Each agent should have **4-5 tools maximum**. Giving an agent 18 tools degrades selection reliability – Claude may pick the wrong tool, call unnecessary tools, or struggle to reason about which to use.

Solution for complex systems: Distribute tools across specialized subagents.

Instead of:
One agent with 18 tools

Do:
Coordinator with: Agent, AskUser
Search subagent with: search_web, search_docs, search_db
Analysis subagent with: analyze_text, extract_entities, summarize
Action subagent with: send_email, create_ticket, update_record

This is the exam's answer, and the underlying principle (one agent, one focused job) is still correct. The current numbers from Anthropic's docs are worth knowing alongside it.

Beyond the Exam Guide: Tool Search

Anthropic's current published guidance is that **tool-selection accuracy degrades once an agent exceeds roughly 30–50 available tools**, and that a typical multi-server MCP setup (GitHub, Slack, Sentry, Grafana, Splunk) burns ~55k tokens of context in tool definitions before any work happens. The fix is no longer "split into subagents" alone – it's the **tool search tool**, which is GA on the Claude API.

```
"tools": [
  { "type": "tool_search_tool_regex_20251119", "name": "tool_search_tool_regex" },
  { "name": "get_weather", "description": "...", "input_schema": { ... },
    "defer_loading": true }
]
```

How it works:

1. You still send **every** tool definition on every request. `defer_loading` controls what enters the *context window*, not what you transmit.
2. Non-deferred tools load into context immediately; deferred ones don't.
3. When Claude needs something else, it calls the search tool. The API returns `tool_reference` blocks and expands them into full definitions inline – the system-prompt prefix is untouched, so **prompt caching survives**.
4. Two variants: `tool_search_tool_regex_20251119` (Claude writes Python `re.search()` patterns, max 200 chars) and `tool_search_tool_bm25_20251119` (natural-language queries, max 500 chars). Both search names, descriptions, argument names, and argument descriptions.

Rule	Detail
Never defer everything	At least one tool must have <code>defer_loading: false</code> , normally the search tool itself. Otherwise: 400, "All tools cannot be deferred"
Keep a hot set	Leave your 3–5 most-used tools non-deferred so Claude can call them without a search round-trip
Limits	Up to 10,000 deferred tools per request; searches return 5 matches by default (Claude may set <code>limit</code> , 1–10,000)
Cache	A tool with <code>defer_loading: true</code> can't also carry <code>cache_control</code> – 400. Put the breakpoint on a non-deferred tool
Composability	Works with <code>strict: true</code> ; the grammar builds from the full toolset
MCP	Don't set <code>defer_loading</code> per tool – set it on the <code>mcp_toolset</code> entry's <code>default_config</code>

When to use it: 10+ tools, tool definitions over 10k tokens, accuracy dropping as the toolset grows, or aggregating multiple MCP servers. **When not to:** fewer than 10 tools, every tool used every request, or definitions totalling under ~100 tokens.

Reconciling the two: the exam's "4-5 tools" is about how many tools are *in front of the model at decision time*. Tool search doesn't raise that number – it keeps it at 3–5 while letting the catalog behind it grow to thousands.

tool_choice Parameter

Controls how Claude interacts with tools:

Value	Behavior	Use Case
<code>"auto"</code> (default)	Claude may call a tool OR respond with text	General conversation with optional tool use
<code>"any"</code>	Claude MUST call a tool, but can choose which	Guarantee structured output
<code>{"type": "tool", "name": "specific_tool"}</code>	Claude MUST call this exact tool	Force a specific operation first
<code>"none"</code>	Claude cannot call any tool; text only	Force a text answer while keeping tool definitions in context

Any `tool_choice` value can also carry `"disable_parallel_tool_use": true` to cap Claude at a single tool call per response (by default it may emit several `tool_use` blocks at once).

Fable 5.1 / Mythos 5.1 (since 2026-09-01) and Opus 5.5 (since 2026-09-22): forced tool use is gone. `{"type": "any"}` and `{"type": "tool", "name": ...}` return `400 invalid_request_error` (`tool_choice: type "tool" and "any" are not supported for this model`) – on `count_tokens` and in the Batches API too. `auto` and `none` still work, and `disable_parallel_tool_use` still works with `auto`. The replacements, in order of preference:

- `tool_choice: {"type": "auto"}` plus an explicit instruction naming the tool (in the `user` turn, or in a mid-conversation `role: "system"` message when the app requires the call), with `strict: true` on the tool so the arguments still match your schema.
- Structured outputs (`output_config.format`, Domain 4 §4.3) when the forced call only existed to get JSON back.

Sonnet 5, Opus 5 (now legacy), and the 4.x family keep all four `tool_choice` values, so the exam's "`any` guarantees a tool call" answer is still correct – but it no longer holds on either current flagship. Check `client.models.retrieve(id).capabilities` before assuming.

When to Use Each

- `"auto"` – Most conversations. Claude decides whether a tool is needed.
- `"any"` – When you need guaranteed structured output. Example: You have multiple extraction schemas (invoice, receipt, contract) and you want Claude to pick the right one and always return structured data.
- Forced specification** – When a specific tool must run first. Example: Always call `extract_metadata` before any enrichment step.
- `"none"` – When a turn must produce prose (e.g., a final summarization pass) without stripping the tool definitions from the request.

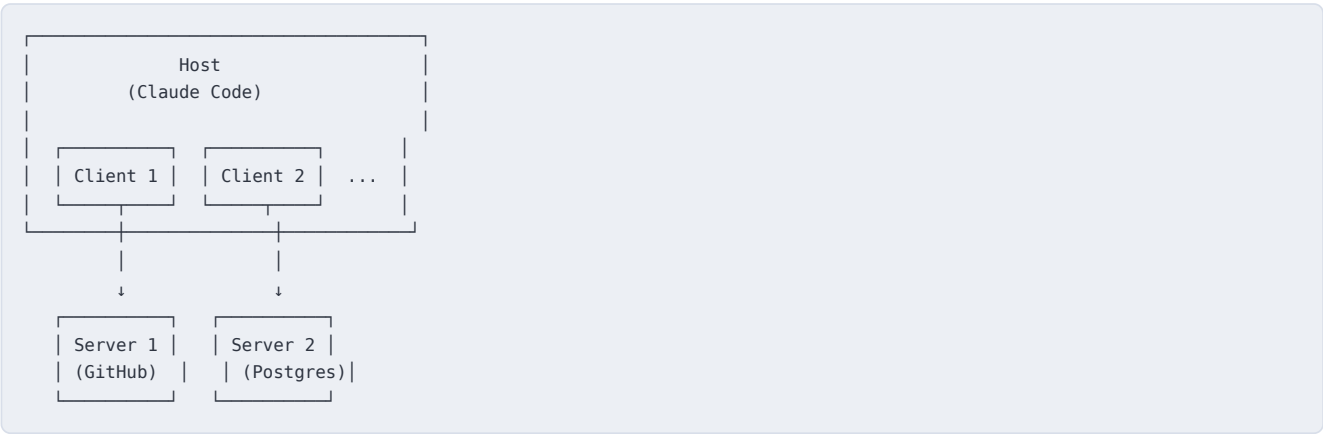
```
# Force Claude to call a specific tool (Sonnet 5 / Opus 5 / 4.x - 400 on Opus 5.5 and Fable 5.1)
response = client.messages.create(
    model="claude-sonnet-5",
    messages=[...],
    tools=[...],
    tool_choice={"type": "tool", "name": "extract_metadata"}
)
```

2.4 MCP Server Configuration

What is MCP?

The Model Context Protocol is an open standard for connecting AI models to external data sources and tools. It provides a standardized way for Claude to interact with databases, APIs, file systems, and other services.

Architecture



- **Host:** The AI application (Claude Code, an IDE) that coordinates MCP clients
- **Client:** A component within the host that maintains a 1:1 connection to an MCP server
- **Server:** A program that provides context (tools, resources, prompts) to the client

Transport Types

Transport	<code>type</code> in config	How It Works	When to Use
stdio	<code>"stdio"</code> (implied when the entry has <code>command</code>)	Standard input/output	Local process communication. No network overhead. Most common for local tools.
Streamable HTTP	<code>"http"</code> (alias: <code>"streamable-http"</code>)	HTTP POST for client-server, optional SSE for streaming	Remote servers, cloud-hosted tools. The only remote transport that supports OAuth
SSE (legacy)	<code>"sse"</code>	Server-Sent Events endpoint	Services that still expose only an SSE endpoint
WebSocket	<code>"ws"</code>	Persistent bidirectional connection	Remote servers that push events unprompted. Header-only auth, no OAuth

Config gotcha: an entry with a `url` but no `type` is a configuration error – Claude Code reads a typeless entry as a stdio server, skips it, and reports `has a "url" but no "type"`.

Always set `type` on remote servers.

MCP Protocol Details

- Uses **JSON-RPC 2.0** message format
- Lifecycle:
 1. `initialize` – Capability negotiation (what the server supports)
 2. `notifications/initialized` – Server confirms ready
 3. `tools/list` – Client discovers available tools
 4. `tools/call` – Client invokes a tool

Claude Code negotiates the **2026-07-28** protocol revision with direct HTTP servers by default on every provider (v2.1.274+; `MCP_PROTOCOL_NEGOTIATION=legacy` opts out). That revision adds URL-mode elicitation: a server can ask the client to open a browser-based flow instead of a form.

Server Primitives

Primitive	What It Is	Example
Tools	Executable functions	<code>search_database</code> , <code>create_issue</code>
Resources	Data sources (read-only content catalogs)	File listings, database schemas
Prompts	Interaction templates	Pre-built prompt workflows

Client Primitives

Primitive	What It Is
Sampling	Server requests an LLM completion from the client
Elicitation	Server requests user input from the client
Roots	Server asks the client which URIs or filesystem boundaries it may operate within

Installation Scopes

Three scopes, and the file a server lands in depends on the scope – not on which file you happened to edit.

Scope	Loads in	Shared with the team?	Stored in
Local (default)	Current project only	No	<code>~/.claude.json</code> , keyed by project path
Project	Current project only	Yes, via version control	<code>.mcp.json</code> in the project root
User	All your projects	No	<code>~/.claude.json</code>
Managed (admin)	Every session in the org	Yes, pushed by IT	<code>managedMcpServers</code> in managed settings (v2.1.259+) – org-provided servers users can't remove

Note the naming trap: MCP *local scope* lives in `~/.claude.json` (home directory), while general *local settings* live in `.claude/settings.local.json` (project directory). They are unrelated files.

Adding servers from the CLI:

```
# Remote HTTP server, shared with the team
claude mcp add --transport http --scope project shared-api https://example.com/mcp

# Remote server with a static auth header
claude mcp add --transport http secure-api https://api.example.com/mcp \
  --header "Authorization: Bearer $TOKEN"

# Local stdio server – everything after -- is passed to the server untouched
claude mcp add airtable --env AIRTABLE_API_KEY=YOUR_KEY -- npx -y @example/mcp-server

# Paste a config block written for another MCP client
claude mcp add-json example '{"type":"http","url":"https://mcp.example.com/mcp"}'
```

Project-Level Configuration (`.mcp.json`)

This file lives in the project root and is version-controlled:

```
{
  "mcpServers": {
    "shared-api": {
      "type": "http",
      "url": "https://example.com/mcp",
      "headers": { "Authorization": "Bearer ${API_TOKEN}" },
      "timeout": 600000
    },
    "postgres": {
      "command": "npx",
      "args": [ "@modelcontextprotocol/server-postgres" ],
      "env": {
        "DATABASE_URL": "${DATABASE_URL}"
      }
    }
  }
}
```

Key details:

- `${DATABASE_URL}` – Environment variable expansion. The actual secret is NOT in the config file. Use `${VAR:-default}` when the variable may be unset.
- `command` + `args` – How to start the MCP server process (stdio transport)
- `timeout` – per-server tool execution timeout in milliseconds; overrides `MCP_TOOL_TIMEOUT` for that server
- Claude Code **prompts for approval** before using project-scoped servers from `.mcp.json` in interactive sessions. `claude -p`, Agent SDK sessions, and cloud sessions can't show that prompt and load them without asking – use `disabledMcpjsonServers` or `--setting-sources` to keep a server out of headless runs; `CLAUDE_CODE_MCP_STARTUP_WAIT_MS` bounds how long the first non-interactive turn waits for servers to connect (`0` = don't wait)
- Some server names are reserved (`workspace`, `claude-in-chrome`, `computer-use`, `Claude Preview`, `Claude Browser`) and will be skipped with a warning

Authentication for Remote Servers

HTTP and SSE servers support **OAuth** – run `/mcp` in-session (or `claude mcp add` then authenticate) and Claude Code handles the browser flow and token refresh. For services without OAuth, pass a static token via `headers`, or generate one at connect time with `headersHelper` (a command whose stdout becomes the header value) when the credential is short-lived. WebSocket servers are header-only.

Tool Search Is On by Default in Claude Code

MCP tool definitions are **deferred** by default: only tool names and server instructions load at session start, and Claude searches for the rest on demand. That is why Claude Code imposes no per-server tool cap – the practical limit is your context budget.

ENABLE_TOOL_SEARCH	Behavior
(unset)	All MCP tools deferred, loaded on demand
true	Force deferral (sends the beta header even through proxies)
auto / auto:N	Load upfront while deferred definitions total under 10% (or N%) of the context window; defer once past it
false	Load every MCP tool upfront

Set `"alwaysLoad": true` on a server entry to exempt it from deferral when Claude needs its tools on every turn. Server authors should write good **server instructions** – with tool search on, those instructions are how Claude decides whether to search your server at all. Claude Code truncates tool descriptions and server instructions at 2,048 characters each (`CLAUDE_CODE_MAX_MCP_DESCRIPTION_LENGTH` changes the cap, v2.1.280+).

Output Limits

Claude Code warns when MCP tool output exceeds **10,000 tokens** and truncates at **25,000 tokens** by default. Raise the ceiling with `MAX_MCP_OUTPUT_TOKENS`; the warning threshold is fixed. This is the mechanical reason tool-output trimming (Domain 5) matters – a chatty server can silently lose the tail of its own response.

MCP Resources

Resources expose content catalogs to reduce exploratory tool calls. Instead of Claude blindly searching, it can browse available resources:

```
{
  "resources": [
    {
      "uri": "file:///project/docs/api-reference.md",
      "name": "API Reference",
      "mimeType": "text/markdown"
    }
  ]
}
```

2.5 Built-in Tools

Claude Code's built-in tools and when to use each:

Tool	Purpose	When to Use
Grep	Search file contents for patterns	Finding function definitions, error messages, imports, string patterns
Glob	Find files by name/extension	Locating files: <code>**/*.test.ts</code> , <code>src/**/*.config.*</code>
Read	Load full file contents	Reading a specific file after finding it with Grep/Glob
Write	Create new files	Creating new files that don't exist
Edit	Targeted modifications	Changing specific sections of existing files (uses unique text matching)
Bash	Run terminal commands	git, npm, running scripts, system commands
WebSearch / WebFetch	Search the web; fetch and read a URL	Looking up current docs, checking an API's behavior
Agent	Spawn a subagent	Delegating bounded or noisy work (see Domain 1 §1.3)
Skill	Invoke a skill by name	Running a packaged workflow
NotebookEdit	Edit Jupyter notebook cells	<code>.ipynb</code> files
AskUserQuestion	Ask the user a blocking question	Decisions only the user can make
ToolSearch	Load a deferred tool's schema on demand	The client-side counterpart of the API's tool search – MCP and rarely used built-in tools ship as names only until Claude searches for them
SendMessage / ListAgents	Message a running or completed subagent, teammate, or another session	Resuming a subagent with its context intact (Domain 3 §3.4); cross-session messaging
Workflow	Run a dynamic-workflow script that orchestrates many subagents	Only when the user opts in ("use a workflow", <code>ultracode</code>) – see Domain 1 §1.3

Codebase Exploration Pattern

The correct pattern for exploring an unfamiliar codebase:

1. Grep for entry points (main functions, route definitions, exports)
2. Read the entry point files
3. Follow imports – Grep for referenced modules
4. Read those files
5. Build understanding iteratively

Anti-pattern: Reading all files upfront. This wastes context and doesn't build understanding.

When Edit Fails

The Edit tool works by matching unique text strings. If the text isn't unique in the file, the edit fails.

Fallback: Use Read to get the full file, then Write to replace the entire file with the modified version.

2.6 Anthropic-Defined and Server-Side Tools

Not every tool is one you write. Some are **Anthropic-defined** (schema-less: you declare a `type` and a `name`, and Claude knows the interface), and some are **server-side** (they execute on Anthropic's infrastructure – results come back as content blocks in the same response, with no client-side execution loop).

Tool	type	Client- or server-side	Result block
Web search	<code>web_search_20260209</code>	Server	<code>web_search_tool_result</code>
Web fetch	<code>web_fetch_20260209</code>	Server	<code>web_fetch_tool_result</code>
Code execution	<code>code_execution_20260521</code>	Server	<code>bash_code_execution_tool_result</code> (<code>.content.stdout</code>)
Tool search (regex / BM25)	<code>tool_search_tool_regex_20251119</code> / <code>..._bm25_20251119</code>	Server	<code>tool_search_tool_result</code>
Memory	<code>memory_20250818</code>	Client (you implement the file ops)	standard <code>tool_result</code>
Bash	<code>bash_20250124</code>	Client	standard <code>tool_result</code>
Text editor	<code>text_editor_20250728</code>	Client	standard <code>tool_result</code>

Details worth carrying into an exam or a design review:

- **Version the type string.** `web_search_20260209` / `web_fetch_20260209` add dynamic filtering and need Opus 4.6+ / Sonnet 4.6+; older models use the basic `web_search_20250305` / `web_fetch_20250910`. Don't also declare `code_execution` alongside the `_20260209` variants – they run code under the hood, and a second execution environment confuses the model.
- **Web fetch only fetches URLs already present in the conversation.** It is not a crawler.
- **Server-tool errors do not raise.** They return HTTP 200 with an error object inside the result block (e.g. `{"error_code": "max_uses_exceeded"}`). For web search, a success `content` is a *list* and an error `content` is an *object* – branch on that before indexing. This is the same "empty vs failed" distinction from §2.2, at the API layer.
- **pause_turn** is how a server-side tool loop tells you it hit its internal cap. Send the response back to continue (Domain 1 §1.1).
- **Bash and text editor are schema-less.** Declaring a custom tool of your own named `"bash"` with an `input_schema` creates a *different* tool.
- **Advisor tool:** its `model` must be at least as capable as the request's top-level `model` (e.g. executor `claude-sonnet-5` → advisor `claude-opus-5`). An invalid pair returns 400.

Parallel Tool Use

By default Claude may emit several `tool_use` blocks in one assistant message. Execute them concurrently and return **all** `tool_result` blocks in a **single** user message. Splitting them across multiple messages silently teaches Claude to stop making parallel calls. For a tool that failed, return its `tool_result` with `is_error: true` – never drop it.

Programmatic Tool Calling

Claude can call *your* custom tool from inside the code execution sandbox instead of round-tripping through the conversation – useful when a tool returns large results that would otherwise flood the context. Declare `{"type": "code_execution_20260120", "name": "code_execution"}` and set `"allowed_callers": ["code_execution_20260120"]` on the custom tool. When responding to a pending programmatic call, the user message must contain **only** `tool_result`

blocks – no text. Not compatible with `strict: true`, `disable_parallel_tool_use`, forced `tool_choice`, or MCP tools.

MCP from the API Side: the MCP Connector

Domain 2's MCP material is written from Claude Code's perspective (a host that manages its own clients). The Messages API can also connect to remote MCP servers directly. It needs **both halves** – passing `mcp_servers` alone is a validation error:

```
client.beta.messages.create(
    model="claude-opus-5-5",
    betas=["mcp-client-2025-11-20"],
    mcp_servers=[{"type": "url", "url": "https://mcp.example.com/mcp", "name": "example"}],
    tools=[{"type": "mcp_toolset", "mcp_server_name": "example"}],
    messages=[...],
)
```

Set `defer_loading` once on the `mcp_toolset` entry's `default_config` (or per tool in `configs`) rather than on individual tool definitions.

Changing Tools Mid-Conversation Without Breaking the Cache

The `tools` array sits *earlier* in the cached prefix than `system`, so editing it between turns invalidates the cache for the whole conversation (Domain 4 §4.8). On the models that support mid-conversation system messages (Fable 5 / 5.1, Mythos 5 / 5.1, Opus 5.5, Opus 5, Opus 4.8 – not Sonnet 5), declare the full tool set up front and then *offer or withdraw* tools with `tool_addition` / `tool_removal` blocks inside a `role: "system"` message (beta header `mid-conversation-tool-changes-2026-07-01`; each block's `tool` is a `tool_reference` by name, or `mcp_tool_reference` / `mcp_toolset_reference` for connector tools – naming a tool that isn't in `tools` is a 400, `tool_reference_unresolved`). A tool declared with `defer_loading: true` stays withheld until a `tool_addition` surfaces it. On Fable 5.1 and Opus 5.5 this is also the only history-safe way to change tools, because rebuilding `tools` counts as editing earlier turns and invalidates later thinking blocks.

Defining Tools in a Message (beta `inline-tools-2026-09-15`, since 2026-09-22)

The reference form assumes you knew every tool at the first request. The inline form lets a `tool_addition` block carry the **full definition** instead:

```
{
  "role": "system",
  "content": [{
    "type": "tool_addition",
    "tool": {
      "type": "tool_definition",
      "definition": {
        "name": "db_query",
        "description": "Run a read-only SQL query against the analytics database.",
        "input_schema": {"type": "object", "properties": {"sql": {"type": "string"}}, "required": ["sql"]}
      }
    }
  ]
}
```

Send a different definition under the **same name** and it replaces the earlier one from that point on – that is how you change a schema, or move a server tool to a newer version, without touching `tools` or the prompt cache (reusing the name of a different *type* of tool is a 400, `tool_name_conflict`). With the `mcp-client-2026-09-15` header as well, the definition can be an `mcp_toolset`; the response then opens with one `mcp_tool_listing` block per server it

fetches – send it back to pin that tool list, and don't read `content[0]` blindly. It works on every model that supports mid-conversation tool changes, Opus 5.5 included.

Rules of thumb: declare what you know up front (in `tools`, deferred if the model shouldn't see it yet) and define inline only what is unknown at the first request or changes later; keep at least one non-deferred tool in `tools`, or the first inline definition changes the start of the rendered prompt and costs a full cache miss; `cache_control` goes on the block or in the definition, not both, and a deferred definition can't carry one. Limits: 10,000 deferred or message-defined tools, and 4 MB of definitions sent after the first user message (`available_tools_limit_exceeded`). After an on-demand compaction, tool changes inside the summarized range carry over only when the compaction request also sent this header – the block then records their net effect in a `tool_changes` field (Domain 5 \$5.7).

Domain 2 Practice Questions

Q1: An agent has 18 tools configured and is frequently selecting the wrong tool. What is the most effective solution?

- A) Improve all 18 tool descriptions to be more detailed
- B) Distribute the tools across specialized subagents with 4-5 tools each
- C) Set `tool_choice` to "any" to force tool usage
- D) Add examples to the system prompt showing correct tool selection

Answer: B – The 4-5 tool maximum per agent is the key architectural principle. Even with better descriptions, 18 tools degrades selection reliability.

Q2: A search tool returns an empty array. What should the agent communicate to the user?

- A) "The search service is currently unavailable"
- B) "No results were found matching your query"
- C) Check the response status to distinguish between "no results" and "search failed"
- D) Retry the search with different parameters

Answer: C – The agent must distinguish between a valid empty result (status 200, no matches) and a failed search (error/timeout). These require fundamentally different responses.

Q3: Where should MCP server credentials be stored in a project using `.mcp.json`?

- A) Directly in the `.mcp.json` file
- B) In environment variables referenced via `${VAR_NAME}` syntax
- C) In a separate `.mcp-secrets.json` file
- D) In the project's `package.json`

Answer: B – `.mcp.json` supports environment variable expansion. Credentials should never be committed to version control.

Q4: A platform team aggregates six MCP servers, exposing about 200 tools. Tool definitions consume ~55k tokens before any work starts, and tool selection has become unreliable. What is the current recommended fix?

- A) Raise `max_tokens` so there is room for the definitions
- B) Enable the tool search tool and mark all but 3-5 frequently used tools `defer_loading: true`
- C) Set `tool_choice: "any"` so Claude is forced to commit to a tool
- D) Set `defer_loading: true` on every tool including the search tool

Answer: B – Tool search keeps only a small hot set plus the search tool in context and loads the rest on demand via `tool_reference` blocks, cutting definition tokens by ~85% while preserving prompt caching. D is a 400 error: at least one tool must stay non-deferred. Splitting across subagents (§2.3) is still valid, but tool search is what makes a 200-tool catalog workable inside one agent.

Q5: An extraction service forces its schema with `tool_choice: {"type": "tool", "name": "extract_invoice"}` and is being moved from Claude Opus 5 to Claude Fable 5.1. What happens, and what is the right fix?

- A) Nothing changes – forced tool use works on every current model
- B) The request returns 400; switch to `tool_choice: "any"` instead
- C) The request returns 400; use `tool_choice: "auto"` with an explicit instruction and `strict: true`, or structured outputs via `output_config.format`
- D) The call silently falls back to text output

Answer: C – Fable 5.1 and Mythos 5.1 reject both forced forms (`any` and `tool`) with a 400. `auto` plus an instruction keeps the tool call, `strict: true` keeps the arguments schema-valid, and structured outputs replace the pattern entirely when the tool only existed to return JSON. Sonnet 5 and Opus 5 still accept forced tool use, but Opus 5.5 rejects it exactly as Fable 5.1 does – so this is a per-model check, not a global rule, and neither current flagship supports it.

Domain 3: Claude Code Configuration & Workflows (20%)

Covers CLAUDE.md hierarchy and loading, custom commands and skills, path-specific rules, plan mode vs direct execution, iterative refinement, and CI/CD integration.

3.0 Surfaces: Where Claude Code Runs

Claude Code is no longer just a CLI. Every surface connects to the **same engine**, so CLAUDE.md files, settings, skills, and MCP servers work identically across all of them – which is the architecturally relevant point: configuration is portable, and so is a session.

Surface	What it is	Install / entry point
Terminal	Full-featured CLI	<code>curl -fsSL https://claude.ai/install.sh bash</code> , then <code>claude</code>
VS Code / Cursor	Extension with inline diffs, @-mentions, plan review	Extensions marketplace
JetBrains	Plugin for IntelliJ, PyCharm, WebStorm (requires the CLI)	JetBrains Marketplace
Desktop app	Standalone app: visual diffs, parallel sessions, scheduled tasks, cloud sessions	macOS / Windows download
Web	Browser sessions with no local setup; long-running tasks on repos you don't have locally	claude.ai/code
Mobile	iOS / Android app for the same cloud sessions	Claude app

Session Portability

Goal	Mechanism
Start locally, finish in the cloud	<code>claude --cloud "<task>"</code> (<code>--remote</code> is a deprecated alias)
Pull a web/cloud session into the terminal	<code>claude --teleport</code>
Run a task as a local background session	<code>claude --bg "<task>"</code> , then <code>claude agents</code> / <code>claude attach <id></code> / <code>logs</code> / <code>stop</code> / <code>respawn</code> / <code>rm</code>
Copy the current session into a background session	<code>/fork [prompt]</code> (a forked <i>subagent</i> that reports back is <code>/subtask</code> ; <code>/branch</code> switches you into a copy)
Hand the current terminal session to the desktop app	<code>/desktop</code>
Drive a running local session from a phone or another browser – or fork it from the Claude app into a local background session (v2.1.273+)	Remote Control
Push external events (Telegram, Discord, iMessage, webhooks) into a session	Channels
Route work from team chat	<code>@Claude</code> in Slack

Scheduling

Three distinct mechanisms – the exam-relevant distinction is *where they run*:

Mechanism	Runs	Use for
Routines (<code>/schedule</code> , web, or desktop)	In the cloud – keeps running with your machine off; can also trigger on API calls or GitHub events	Morning PR reviews, overnight CI failure analysis, weekly dependency audits
Desktop scheduled tasks	On your machine, with local file and tool access	Anything that needs the local filesystem or private network
<code>/loop</code>	Inside the current CLI session	Quick polling within one sitting – omit the interval and Claude self-paces between iterations

3.1 CLAUDE.md Hierarchy

What is CLAUDE.md?

CLAUDE.md files are instruction files that tell Claude Code how to behave in a specific context. They're like `.editorconfig` or `.eslintrc` but for an AI agent – defining coding standards, project conventions, tool restrictions, and workflow rules.

Loading Order (Broader → More Specific)

Claude Code loads CLAUDE.md files in a hierarchy. More specific files override broader ones:

1. Managed Policy (org-wide, admin-controlled)

└─ /etc/claude-code/CLAUDE.md (Linux, WSL)

└─ /Library/Application Support/ClaudeCode/CLAUDE.md (macOS)

└─ C:\Program Files\ClaudeCode\CLAUDE.md (Windows)

2. User-Level (personal, NOT version-controlled)

└─ ~/.claude/CLAUDE.md

3. Project-Level (shared via version control)

└─ ./CLAUDE.md or ~/.claude/CLAUDE.md

4. Local project (personal, gitignored)

└─ ./CLAUDE.local.md

5. Directory-Level (loaded on demand)

└─ ./src/api/CLAUDE.md (loaded when Claude reads files in src/api/)

└─ ./tests/CLAUDE.md (loaded when Claude reads files in tests/)

Key distinction: Project-level CLAUDE.md is committed to git and shared with the team. User-level is personal and local. `CLAUDE.local.md` is personal per project (gitignore it). Directory-level loads dynamically when Claude works in that directory.

AGENTS.md Interop

Since v2.1.277, a project with **no** `CLAUDE.md` falls back to reading `AGENTS.md` (on every provider since v2.1.281; toggle under "Project instructions" in `/config`). When both exist, `CLAUDE.md` wins and `AGENTS.md` is ignored during normal sessions – `/init` still reads it when scaffolding. If your repo already uses `AGENTS.md` for other coding agents and you also need Claude-specific rules, keep one source of truth by importing it:

```
# CLAUDE.md
@AGENTS.md

## Claude-specific overrides
Use plan mode for changes under `src/billing/`.
```

@import Syntax

CLAUDE.md files can import other files for modular configuration:

```
# Project Configuration

@docs/coding-standards.md
@docs/api-conventions.md
@~/ .claude/my-personal-preferences.md

See @README.md for project overview.
```

Resolution rules:

- Relative paths resolve relative to the file containing the import
- `@~/` resolves to the user's home directory
- Maximum import depth: 5 hops (to prevent circular imports)
- Imported files can themselves contain @imports (up to depth limit)

.claude/rules/ Directory

For topic-specific rules organized as separate files:

```
.claude/
rules/
  testing.md      - Testing conventions
  api-design.md   - API endpoint patterns
  security.md     - Security requirements
  typescript.md   - TypeScript-specific rules
```

Without frontmatter: Rules load unconditionally at session start. **With frontmatter:** Rules load conditionally based on file paths.

Auto Memory

Claude Code maintains automatic memory at:

```
~/ .claude/projects/<project>/memory/MEMORY.md
```

- **First 200 lines OR 25 KB** (whichever comes first) loaded at session start
- Machine-local (not version controlled)
- All worktrees in the same git repo share one memory directory
- Claude writes notes here automatically about project patterns, decisions, preferences
- Requires Claude Code v2.1.59 or later
- `autoMemoryEnabled: false` disables it; `autoMemoryDirectory` relocates it
- Topic files (e.g., `debugging.md`, `api-conventions.md`) alongside `MEMORY.md` are loaded on demand – the 200-line cap applies only to `MEMORY.md` itself
- Current layout: `MEMORY.md` is an **index** (one line per memory) and each memory is its own file with YAML frontmatter – `name`, `description`, and a `type` of `user` / `feedback` / `project` / `reference`; Claude Code stamps a `modified` timestamp on every write (v2.1.214+)
- Subagents can maintain their own auto memory

Monorepo and Multi-Directory Controls

Setting / flag	Purpose
<code>claudeMdExcludes</code> (in settings)	Glob-list of CLAUDE.md files to skip – useful when another team's nested CLAUDE.md would otherwise load
<code>--add-dir <path></code>	Grant Claude access to additional working directories
<code>CLAUDE_CODE_ADDITIONAL_DIRECTORIES_CLAUDE_MD=1</code>	Also load CLAUDE.md / rules from <code>--add-dir</code> paths
<code>CLAUDE_CODE_NEW_INIT=1</code>	Interactive multi-phase <code>/init</code> that scaffolds CLAUDE.md, skills, and hooks

3.2 Skills (and Legacy Slash Commands)

Major change (2026): Custom slash commands have been merged into skills. A file at `.claude/commands/deploy.md` and a skill at `.claude/skills/deploy/SKILL.md` both create `/deploy` and behave the same way. The old `/project:name` and `/user:name` namespacing is gone – commands are just `/name`, and higher-priority locations win when names collide.

Skill Locations (Priority: Enterprise > Personal > Project > Plugin)

Location	Path	Applies to
Enterprise	Managed settings path	All users in the org
Personal	<code>~/.claude/skills/<name>/SKILL.md</code>	All your projects
Project	<code>.claude/skills/<name>/SKILL.md</code>	This project only
Plugin	<code><plugin>/skills/<name>/SKILL.md</code>	Where the plugin is enabled (namespaced <code>plugin-name:skill-name</code>)

Legacy `.claude/commands/*.md` still works and supports the same frontmatter – but skills add a directory for supporting files, dynamic context injection, and subagent execution. When a skill and a command share a name, the skill wins. Claude Code skills follow the open [Agent Skills](#) standard.

Example SKILL.md

```
---
name: deploy
description: Deploy the application to a specified environment
context: fork
agent: Explore
allowed-tools: Bash(git *) Bash(npm run deploy *)
argument-hint: [environment]
disable-model-invocation: true
---

# Deploy Skill

Deploy the application to $ARGUMENTS.

1. Run the test suite
2. Build the application
3. Push to the deployment target
4. Verify the deployment succeeded
```

Frontmatter Reference (Current)

Field	Purpose
<code>name</code>	<code>/slash-command</code> name (defaults to directory name; lowercase, digits, hyphens, max 64 chars)
<code>description</code>	When to use the skill – Claude uses this to decide whether to auto-load. Capped at 1,536 chars combined with <code>when_to_use</code>
<code>when_to_use</code>	Additional trigger phrases / example requests, appended to <code>description</code>
<code>argument-hint</code>	Hint shown in autocomplete, e.g. <code>[issue-number]</code>
<code>arguments</code>	Named positional arguments for <code>\$name</code> substitution
<code>disable-model-invocation</code>	<code>true</code> → only the user can invoke via <code>/name</code> ; Claude can't auto-trigger
<code>user-invocable</code>	<code>false</code> → hidden from <code>/</code> menu; Claude can still invoke (background knowledge)
<code>allowed-tools</code>	Tools pre-approved while this skill is active
<code>model</code>	Model override for this skill's turn
<code>effort</code>	<code>low</code> / <code>medium</code> / <code>high</code> / <code>xhigh</code> / <code>max</code> – effort-level override
<code>context</code>	<code>fork</code> runs the skill in an isolated subagent
<code>agent</code>	Which subagent type executes when <code>context: fork</code> (e.g. <code>Explore</code> , <code>Plan</code> , <code>general-purpose</code>)
<code>hooks</code>	Hooks scoped to this skill's lifecycle
<code>paths</code>	Glob patterns – auto-load only when working with matching files
<code>shell</code>	<code>bash</code> (default) or <code>powershell</code> for inline command injection

Substitution Variables

Variable	Resolves To
<code>\$ARGUMENTS</code>	Full argument string passed by the user
<code>\$ARGUMENTS[N]</code> / <code>\$N</code>	0-indexed positional argument (<code>\$0</code> , <code>\$1</code> , ...)
<code>\$name</code>	Named argument declared in <code>arguments:</code> frontmatter
<code>\${CLAUDE_SESSION_ID}</code>	Current session identifier
<code>\${CLAUDE_SKILL_DIR}</code>	Directory containing the SKILL.md file

Dynamic Context Injection

Skills can preprocess shell output and file contents before Claude sees them:

```
---
name: pr-summary
description: Summarize changes in a pull request
context: fork
agent: Explore
allowed-tools: Bash(gh *)
---

## PR context
- Diff: !`gh pr diff`
- Comments: !`gh pr view --comments`
- Changed files: !`gh pr diff --name-only`

## Your task
Summarize this pull request...
```

`!`cmd`` runs the command *before* the skill body is sent. Multi-line blocks use `````. File inclusion with `@path` also works. Disable globally with `"disableSkillShellExecution": true`.

Context Isolation (`context: fork`)

- Spawns the skill as a subagent with its own context
- The `agent:` field picks the execution environment (built-in `Explore`, `Plan`, `general-purpose`, or a custom `.claude/agents/*` subagent)
- CLAUDE.md still loads; main conversation history does not
- Useful for noisy operations (deep research, deployments) that would pollute the main context

Skill Lifecycle and Compaction

When invoked, a skill's rendered body enters the conversation as a single message and stays there. Claude Code does **not** re-read the file on later turns – write standing instructions, not one-time steps. Auto-compaction re-attaches the most recently invoked skills (first 5 000 tokens of each, 25 000-token combined budget), dropping older ones if you used many.

Bundled Skills

Claude Code ships with a set of bundled skills that varies by release. Recent versions include:

- `/claude-api` – Build and migrate Claude API / Anthropic SDK apps; subcommands `migrate`, `prompt-audit`, `upgrade` (Python SDK 0.x → 1.x), `cost-optimize`, `preserved-thinking-migration` (find and fix the history edits a harness makes), `build-eval` / `hillclimb`, `managed-agents-onboard`
- `/code-review` – Multi-agent review of pending changes or a PR (`ultra` runs it in the cloud; `/ultrareview` is the legacy alias)
- `/diff` – Review the working-tree changes, including Claude's edits, before shipping
- `/security-review` – Security review of the pending changes on the branch
- `/simplify` – Reuse / simplification / efficiency cleanups on changed code
- `/dataviz` – Charts and dashboards with consistent design
- `/design` – Multi-artboard visual design canvas
- `/init` – Scaffold a CLAUDE.md for the codebase
- `/run` – Launch the project's app to confirm a change works
- `/loop` – Run a prompt or slash command on a recurring interval
- `/schedule` – Create and manage cloud routines
- `/deep-research` – A bundled dynamic *workflow*: fan out web searches, cross-check sources, synthesize a cited report
- `/workflows` / `/workflow-authoring` – Watch running workflows; load the script-writing reference (when dynamic workflows are enabled)
- `/advisor` – Consult a second model (`fable`, `opus`, `sonnet`) at key moments of a task
- `/fast` – Toggle fast mode (Opus 5.5 / Opus 5 / Opus 4.8; premium pricing – \$8 / \$40 on Opus 5.5, \$10 / \$50 on Opus 5 and 4.8 – same model, faster output; also honoured in cloud and self-hosted remote sessions)
- `/update-config` – Configure settings.json, hooks, and permissions
- `/fewer-permission-prompts` – Build a permission allowlist from your usage
- `/output-style [name]` – List and switch output styles, including in headless and Remote Control sessions (v2.1.269+)
- `/insights` – Usage report; now estimates how many recent permission prompts auto mode could have handled
- `/skill-doctor` – Show what each skill costs in context and how often it is used (v2.1.252+); every listed skill costs tokens on every turn

- `/doctor` – Diagnose Claude Code setup problems; `/feedback` sends a report with session context

Check `/help` in your installed version (or the commands page in the docs) for the current list – the set changes release to release.

Plugins

A **plugin** is a distributable bundle of skills, subagents, hooks, and MCP servers, installed from a marketplace and enabled per project. Plugin-provided items are namespaced (`plugin-name:skill-name` for skills, `plugin:<plugin>:<server>` for MCP servers) so they can't collide with your own. Plugins sit at the lowest priority in the skill resolution order (Enterprise > Personal > Project > Plugin).

Plugins are also the unit that `claude plugin eval` tests: you write an eval suite against the plugin's skills and run it in a sandbox or in CI, and it emits a JSON report. That is the packaging answer to "how does a team ship and regression-test its Claude Code conventions?"

Recent plugin tooling: `claude plugin validate` also checks a plugin's `.mcp.json` entries (servers that would be silently dropped, undeclared `${user_config.*}` references, insecure URLs); `--plugin-dir` can point at a folder of plugins and picks up children added while running; `claude plugin install <plugin> --marketplace <source>` offers to add the marketplace on the way in; and skills and plugins enabled on your claude.ai account sync into terminal sessions signed in with it (`syncClaudeAiSkills` / `syncClaudeAiPlugins: false` opt out).

3.3 Path-Specific Rules

Conditional Rule Loading

Rules in `.claude/rules/` can specify which files they apply to using glob patterns in YAML frontmatter:

```
---
paths:
  - "src/api/**/*.ts"
  - "src/api/**/*.test.ts"
---

# API Development Rules

All API endpoints must:
- Use Zod for request validation
- Return standardized error responses
- Include rate limiting middleware
- Have corresponding integration tests
```

```
---
paths:
  - "**/*.test.tsx"
  - "**/*.test.ts"
  - "**/*.spec.ts"
---

# Testing Conventions

- Use React Testing Library, not Enzyme
- Test behavior, not implementation
- Mock external APIs, not internal modules
- Each test file must have a describe block matching the component/function name
```

When Path-Specific Rules Beat Directory CLAUDE.md

Scenario: Testing conventions that apply to test files scattered throughout the entire codebase.

Directory CLAUDE.md approach:

```
src/
  components/
    CLAUDE.md ← Would need testing rules here
    Button.test.tsx
  api/
    CLAUDE.md ← And here
    users.test.ts
  utils/
    CLAUDE.md ← And here
    format.test.ts
```

Path-specific rules approach:

```
.claude/
  rules/
    testing.md ← One file with paths: ["**/*.test.*"]
```

The rules approach is superior because:

- Single source of truth for testing conventions
- Applies everywhere test files exist, even new directories
- No need to duplicate CLAUDE.md files across directories

3.4 Plan Mode vs Direct Execution

Permission Modes (Current)

Plan mode is now one entry in a broader permission-mode system. Start in a specific mode with `--permission-mode <mode>`, or cycle modes with **Shift+Tab** in an interactive session.

Mode	Behavior
default	Standard permission prompts
acceptEdits	Auto-accept file edits; still prompt for other tools
plan	Exploration-only: Claude designs an approach and presents it before touching files
auto	A classifier decides per-tool whether to prompt – see <code>claude auto-mode defaults</code> . Since v2.1.278–282 the classifier runs server-side by default on the Claude API, Bedrock, Vertex, Foundry, and gateways (no classifier overhead billed; <code>CLAUDE_CODE_AUTO_MODE_SERVER=0</code> opts out; <code>/status</code> shows which is active), and read-only or sandboxed commands also wait for its review
dontAsk	Never prompt for the pre-approved list
bypassPermissions	Skip all prompts (dangerous; use only for short, contained tasks)

When to Use Plan Mode

Indicator	Example
Multi-file changes	Refactoring auth across 15 files
Architectural decisions	Choosing between REST and GraphQL
Multiple valid approaches	Several ways to implement caching
Large scope	Library migration affecting 45+ files
Unclear requirements	"Make the app faster" – needs investigation first

In plan mode, Claude:

1. Explores the codebase (Glob, Grep, Read)
2. Identifies affected files and patterns
3. Designs an approach
4. Presents the plan for approval
5. Only implements after user approval

When to Use Direct Execution

Indicator	Example
Single-file fix	Bug fix with clear stack trace
Obvious scope	Adding a null check
Clear instructions	"Add a <code>createdAt</code> field to the User model"
Small, contained	Fixing a typo, updating a constant

Built-in Subagents

Four subagents ship with Claude Code and are registered by default in interactive sessions:

Agent	Purpose	Tools
Explore	Fast file discovery and codebase search	Read-only; skips CLAUDE.md and git status
Plan	Research for plan mode before proposing changes	Read-only; skips CLAUDE.md and git status
general-purpose	Multi-step tasks needing exploration <i>and</i> action	All subagent tools
claude	Catch-all fallback	All subagent tools

The Explore agent is the canonical context-preservation move:

Main conversation context is preserved

↓

Explore subagent runs (Glob, Grep, Read – lots of output)

↓

Returns concise summary to main conversation

↓

Main context stays clean

Without it, a deep codebase exploration would fill the main context window with raw file contents and search results.

Custom Subagents (`.claude/agents/`)

Custom subagents are markdown files with YAML frontmatter, resolved in the order: managed settings > `--agents` CLI JSON > project (`.claude/agents/`) > user (`~/.claude/agents/`) > plugin.

```
---
name: db-reader
description: Execute read-only database queries
tools: Read, Grep, Bash
disallowedTools: Write, Edit
model: sonnet
effort: low
permissionMode: plan
maxTurns: 10
skills: [api-conventions]
mcpServers: [github]
memory: project
isolation: worktree
background: false
omitClaudeMd: true
experimental:
  cacheTtl: 1h
hooks:
  PreToolUse:
    - matcher: "Bash"
      hooks:
        - type: command
          command: ./scripts/validate-readonly-query.sh
---

Execute SELECT queries only. Never issue writes.
```

Three ways to invoke: name it in natural language ("use the db-reader to..."), `@`-mention it to guarantee that specific agent runs, or start a whole session as it with `claude --agent db-reader`.

What a fresh subagent loads: its own system prompt, the delegation prompt, the CLAUDE.md hierarchy (unless `omitClaudeMd: true`), a git status snapshot, preloaded skills, and environment details. **What stays out:** your conversation history, the main session's auto memory, and other subagents' results – which is exactly the "context does not inherit" rule from Domain 1 §1.3.

Constraints worth remembering: permission modes flow *down* – if the parent runs `bypassPermissions`, `acceptEdits`, or `auto`, the subagent inherits it and cannot override. Subagents nest 3 deep by default and 20 can run concurrently. Completed subagents can be resumed by ID or name via `SendMessage`, retaining their prior context; one-shot agents (Explore, Plan) cannot be resumed.

Prompt-injection note: since v2.1.277 a subagent's result reaches the main agent under a header marking it as subagent output, with the result indented, so text inside it can't pass as the session's own instructions (earlier releases scanned reports for instruction-shaped patterns and escaped them). It's a mitigation, not a permission boundary – every tool call still goes through the session's permission checks.

Beyond Subagents: Agent Teams and Dynamic Workflows

Subagents are one worker per delegation, decided turn by turn. Two larger units exist when that isn't enough, and the exam-style question is *who holds the plan*:

	Subagents	Agent teams (experimental)	Dynamic workflows
What it is	A worker Claude spawns	A lead session supervising peer Claude Code sessions with a shared task list and inter-agent messaging	A JavaScript script Claude writes and a runtime executes in the background
Who decides what runs next	Claude, turn by turn	The lead agent	The script (<code>agent()</code> , <code>parallel()</code> , <code>pipeline()</code> , <code>phase()</code>)
Scale	A few delegated tasks per turn	A handful of long-running peers	Dozens to hundreds of agents per run; resumable in-session
Enable	Built in	<code>CLAUDE_CODE_EXPERIMENTAL_AGENT_TEAMS=1</code>	Paid plans and API access; Pro turns it on in <code>/config</code> ; <code>workflowSizeGuideline</code> sets the advisory agent count (medium is now 10 agents, Pro defaults to small); <code>CLAUDE_CODE_WORKFLOW_MAX_CONCURRENT_AGENTS</code> (1–256) raises the per-run concurrency cap

A workflow only runs when the user opts in – the `ultracode` effort level, an explicit "use a workflow", or a bundled workflow skill such as `/deep-research`. `/workflows` watches, pauses, resumes, or saves a run. Codifying the orchestration is the point: the plan becomes a reviewable, rerunnable script instead of a coordinator's memory (Domain 1 §1.2).

Combining Modes

Plan mode for investigation → Direct execution for implementation:

1. Enter plan mode
 2. Explore the codebase
 3. Design the approach
 4. Present plan to user
 5. User approves
 6. Exit plan mode
 7. Implement directly, file by file

3.5 Iterative Refinement Techniques

1. Concrete Examples

When prose instructions produce inconsistent results, provide 2-3 input/output examples:

Instead of: "Format dates consistently"

Use: "Format dates as YYYY-MM-DD. Examples:
Input: 'January 5th, 2024' → Output: '2024-01-05'
Input: '12/31/23' → Output: '2023-12-31'
Input: 'Mar 2024' → Output: '2024-03-01'"

2. TDD Iteration

Write tests first, then iterate by sharing test failures:

1. Write the test: `expect(validateEmail("bad")).toBe(false)`
2. Run tests → FAIL
3. Share failure with Claude
4. Claude fixes the implementation
5. Run tests → PASS or new failures
6. Repeat until green

3. Interview Pattern

Claude asks clarifying questions before implementing:

```
User: "Add authentication to the app"
Claude: "Before I implement, a few questions:
- JWT or session-based?
- Do you need refresh tokens?
- What's the session duration?
- Should I add role-based access control?"
```

4. Multiple vs Independent Issues

Multiple interacting issues: Address in a single message because fixes may conflict.

```
"The auth middleware has two issues:
1. It doesn't check token expiry
2. It returns 500 instead of 401 for invalid tokens
Fix both together."
```

Independent issues: Fix sequentially to keep each change clean and reviewable.

3.6 CI/CD Integration

CLI Flags for CI

Flag	Purpose	Critical?
<code>-p</code> / <code>--print</code>	Non-interactive mode	MUST use in CI or the process hangs waiting for input
<code>--output-format</code>	<code>text</code> , <code>json</code> , or <code>stream-json</code> for machine-parseable output	Needed for downstream processing
<code>--input-format stream-json</code>	Stream input events back into the agent	For pipeline composition
<code>--json-schema '{...}'</code>	Validate final output against a JSON Schema (print mode only)	Enforces CI output shape
<code>--max-turns N</code>	Hard cap on agentic turns; exits with error if hit	Guardrail
<code>--max-budget-usd X</code>	Stop once API spend hits \$X (print mode only)	Cost guardrail
<code>--bare</code>	Skip auto-discovery (hooks, skills, plugins, MCP, auto memory, CLAUDE.md) – fast scripted starts	Low-overhead scripted calls
<code>--no-session-persistence</code>	Don't save the session to disk	Ephemeral CI runs
<code>--fallback-model sonnet</code>	Auto-fall back when the primary model is overloaded	Reliability
<code>--include-hook-events</code>	Emit all hook lifecycle events into the stream	Observability
<code>--include-partial-messages</code>	Emit partial streaming events	Debugging
<code>--permission-mode <mode></code>	Start in <code>default</code> / <code>acceptEdits</code> / <code>plan</code> / <code>auto</code> / <code>dontAsk</code> / <code>bypassPermissions</code>	See §3.4
<code>--permission-prompt-tool <mcp></code>	Delegate permission prompts to an MCP tool in headless mode	Non-interactive approvals
<code>--permission-prompts none</code>	Deny anything that would prompt when nobody can answer (v2.1.259+); the active mode still decides everything else	Unattended runners
<code>--restricted</code>	Remove command/code-running tools and <code>WebFetch</code> , confine file tools to the working directories, load only managed settings, refuse <code>bypassPermissions</code> (v2.1.248+)	Eval harnesses on shared machines
<code>--safe-mode</code>	Start with every customization disabled (CLAUDE.md, skills, hooks, MCP, plugins, workflows, auto memory) – unlike <code>--bare</code> , permissions still apply	Troubleshooting a broken config
<code>--session-id</code> , <code>--resume</code> , <code>--continue</code> , <code>--fork-session</code>	Session lifecycle control	Multi-step pipelines
<code>--setting-sources user,project,local</code>	Restrict which settings layers are loaded	Reproducible runs
<code>--effort <level></code>	<code>low</code> / <code>medium</code> / <code>high</code> / <code>xhigh</code> / <code>max</code> / <code>ultracode</code>	Tune cost vs quality per run
<code>--agent <name></code> / <code>--agents '<json>'</code>	Run the session as a named subagent, or define subagents inline	Specialized CI jobs
<code>--append-subagent-system-prompt</code> / <code>-file</code>	Append text (or a file, v2.1.261+) to every subagent's prompt except forks (non-interactive only)	Consistent citation/format rules
<code>--forward-subagent-text</code>	Emit subagent messages into the stream	Observability across delegated work

<code>--system-prompt</code> / <code>--system-prompt-file</code> / <code>--append-system-prompt</code>	Replace or extend the system prompt (use the <code>-file</code> forms for large prompts – self-hosted runners now pass prompts as files; <code>--system-prompt-snapshot off</code> re-renders it every request while you iterate on the text)	Purpose-built CI personas
<code>--exclude-dynamic-system-prompt-sections</code>	Move per-machine sections out of the system prompt	Prompt-cache reuse across runners
<code>--strict-mcp-config</code>	Use only the servers from <code>--mcp-config</code>	Hermetic MCP surface
<code>--init</code> / <code>--init-only</code>	Run Setup / SessionStart hooks (and exit)	Warm a CI workspace before the real run
<code>--bg</code> / <code>--background</code> + <code>--exec</code>	Start as a background agent; run a shell command as a background job	Long tasks that shouldn't block
<code>--environment ccpool_*</code> / <code>--ref</code>	Target a self-hosted environment; check out a named ref	Fleet / remote execution
<code>--betas</code>	Send beta headers with API requests	Opt into preview features

Example CI Usage

```
# GitHub Actions example
- name: Code Review
  run: |
    claude -p "Review this PR for bugs and security issues" \
      --output-format json \
      --json-schema '{"type":"object","properties":{"issues":{"type":"array"}}}'
```

Managed CI Integrations

You don't always wire the CLI up by hand:

Integration	What it does
GitHub Actions	Run Claude Code in a workflow – PR review, issue triage, <code>@claude</code> mentions
GitLab CI/CD	Same model on GitLab pipelines
GitHub Code Review	Automatic review on every PR, no workflow file to maintain
Routines	Cloud-scheduled runs that can also fire on GitHub events or API calls

The `-p` + `--output-format json` pattern below is still the right primitive when you need full control; these integrations are the managed path.

Cost, Cache, and Fleet Controls (Settings)

A few settings that turn up in "how does the platform team keep 200 developers' sessions cheap and consistent" scenarios:

Setting	Purpose
<code>promptCacheTtl</code> / <code>subagentPromptCacheTtl</code>	Choose the 5-minute or 1-hour prompt-cache lifetime for the session and for subagents (per-agent override: <code>experimental.cacheTtl</code> in frontmatter)
<code>modelPicker</code> / <code>modelPricing</code> (managed)	Curate the <code>/model</code> list; supply custom per-token rates so <code>/cost</code> is accurate behind a gateway (a <code>multiplier</code> up to 10 marks up internal chargeback rates)
<code>managedMcpServers</code> (managed)	Org-provided MCP servers every session gets
<code>bashOutputMaxChars</code>	Cap inline Bash output (up to 128K chars) before it floods the context. <code>taskOutputMaxChars</code> no longer does anything – the TaskOutput tool was removed in v2.1.277 and Claude reads a background task's output file with Read
<code>permissions.blockReadsOutsideWorkingDirectories</code>	Keep file reads inside the working directories
<code>workflowSizeGuideline</code>	Advisory agent-count ceiling for dynamic workflows
<code>effortLevel</code> / <code>maxEffortLevel</code>	Effort is now per model : a level saved for one model doesn't carry to a newly released one (Opus 5.5 starts at its own default until you pick one with <code>/effort</code>), and <code>maxEffortLevel</code> (top-level or per model under <code>modelSettings</code>) caps it on every provider. Claude Code's default Opus is 5.5, and Pro / Team Standard plans now default to Opus rather than Sonnet
<code>"attribution": false</code>	Hide all commit and PR attribution (v2.1.281+; older CLIs skip a settings file that holds it, so keep the object form in files shared across versions)
<code>syncClaudeAiSkills</code> / <code>syncClaudeAiPlugins</code>	Opt out of syncing claude.ai-enabled skills and plugins into terminal sessions

`/cost` now reports per-session prompt-cache statistics and cache-miss diagnostics – the Claude Code view of the same prefix-stability problem described in Domain 4 §4.8.

Message Batches API

For non-blocking, cost-optimized batch operations:

Feature	Detail
Cost savings	50% compared to synchronous API
Processing window	Up to 24 hours, no latency SLA
Correlation	<code>custom_id</code> per request for matching responses – results arrive in any order , so key by <code>custom_id</code> , never by position
Multi-turn	NOT supported within a single batch request
Extended output	Up to 300k output tokens on Opus 5.5 / 5 / 4.8 / 4.7 / 4.6 and Sonnet 5 / 4.6 (not Fable 5.1) with beta header <code>output-300k-2026-03-24</code> (vs 128k synchronous)

Good for:

- Overnight code analysis reports
- Weekly audit summaries
- Nightly test generation
- Batch document processing

NOT good for:

- Pre-merge checks (blocking, needs immediate response)

- Real-time user interactions
- Anything requiring tool calling loops within a single request

Handling Batch Failures

```
# Process batch results
for result in batch_results:
    if result.status == "failed":
        failed_ids.append(result.custom_id)

# Resubmit only failures
resubmit_batch = [r for r in original_requests if r.custom_id in failed_ids]
```

Session Context Isolation for Reviews

Problem: The same Claude session that wrote code is less effective at reviewing it because it retains the reasoning context (it already "knows" why it made each decision, making it less likely to question them).

Solution: Use independent review instances that don't have the generation context.

```
Generation session: Claude writes the code
      ↓
      (new session, no shared context)
      ↓
Review session: Claude reviews the code fresh
```

Domain 3 Practice Questions

Q1: A team wants testing conventions to apply to all `*.test.ts` files throughout their codebase, which has tests scattered across many directories. What is the most maintainable approach?

- A) Add a CLAUDE.md in every directory that contains test files
- B) Create a single rule file in `.claude/rules/` with `paths: ["**/*.test.ts"]` frontmatter
- C) Add testing rules to the project-level CLAUDE.md
- D) Create a `tests/` directory and put all tests there with a CLAUDE.md

Answer: B – Path-specific rules in `.claude/rules/` are superior to directory-level CLAUDE.md when conventions span multiple directories. A single rule file with glob patterns covers all matching files without duplication.

Q2: When should Claude Code use plan mode instead of direct execution?

- A) When fixing a single bug with a clear stack trace
- B) When migrating a library that affects 45+ files across the project
- C) When adding a null check to a function
- D) When updating a configuration constant

Answer: B – Plan mode is for multi-file changes with architectural implications and multiple valid approaches. A library migration affecting 45+ files clearly fits. The other options are small, obvious-scope changes suited for direct execution.

Q3: A CI pipeline needs to run Claude Code for automated code review. Which flag is essential?

- A) `--verbose`

- B) `-p` / `--print`
- C) `--force`
- D) `--no-cache`

Answer: B – The `-p` / `--print` flag enables non-interactive mode, which is mandatory in CI. Without it, Claude Code hangs waiting for user input. Pair it with `--output-format json` (or `stream-json`), `--max-turns` / `--max-budget-USD` for production-grade guardrails, and `--permission-prompts none` when no host can answer a permission prompt.

Domain 4: Prompt Engineering & Structured Output (20%)

Covers writing explicit criteria, few-shot prompting, getting guaranteed structured output via tool_use and JSON schemas, validation-retry loops, batch processing, and multi-pass review architecture.

4.1 Explicit Criteria

The Problem with Vague Instructions

Vague instructions produce inconsistent results because the model interprets them differently each time.

Bad (vague):

```
"Be conservative when flagging issues."  
"Only report high-confidence findings."  
"Use your best judgment on severity."
```

Good (explicit):

```
"Flag a comment as outdated ONLY when the claimed behavior directly  
contradicts the actual code behavior. Do NOT flag comments that are  
merely incomplete or could be more detailed."
```

Explicit Criteria Structure

For each type of judgment the model must make, define:

1. **What qualifies** – Specific conditions that trigger the judgment
2. **What doesn't qualify** – Boundary cases that should NOT trigger it
3. **Examples** – Concrete input/output pairs showing the boundary

```
Flag as SECURITY ISSUE when:  
- User input is passed to SQL queries without parameterization  
- User input is rendered in HTML without escaping  
- Credentials are hardcoded in source files  
  
Do NOT flag:  
- ORM queries (already parameterized)  
- Rendering user input in server-side templates with auto-escaping  
- Credentials loaded from environment variables
```

False Positive Impact

High false positive rates in one category undermine trust in ALL categories, even accurate ones. Developers start ignoring all warnings.

Solution: When a category has high false positives, temporarily disable it and improve the criteria prompts before re-enabling. Don't let one noisy category degrade the value of accurate ones.

4.2 Few-Shot Prompting

When to Use Few-Shot

- Ambiguous scenarios where prose instructions aren't precise enough
- Tasks where the format or reasoning pattern needs demonstration
- Reducing hallucination in extraction tasks
- Showing how to handle edge cases

How Many Examples

2-4 targeted examples is the sweet spot. Too few may not cover the pattern. Too many wastes context and may overfit to the examples.

What Good Examples Include

Each example should demonstrate:

1. **Input** – What the model receives
2. **Output** – What it should produce
3. **Reasoning** – WHY this output is correct (not just what to do, but why)

Example 1:

Input: "// Returns the user's full name"

Code: `function getName(user) { return user.firstName; }`

Output: {

```
"issue": "Comment claims full name but function only returns firstName",  
"severity": "medium",  
"location": "line 1",  
"reasoning": "The comment says 'full name' but the code returns only  
  firstName, not firstName + lastName. This is a factual contradiction."  
}
```

Example 2:

Input: "// Validates the input"

Code: `function validate(input) { return input.length > 0 && input.length < 100; }`

Output: null

Reasoning: "The comment says 'validates input' and the function does validate input. The comment is vague but not wrong – it doesn't claim specific validation rules that contradict the implementation."

Few-Shot Enables Generalization

Good examples teach the pattern, not just the specific cases. The model should generalize to novel situations by understanding the reasoning behind each example.

4.3 Structured Output via `tool_use`

The Problem

When you need Claude to return structured data (JSON), free-form text generation can produce:

- Malformed JSON
- Missing required fields
- Unexpected field names
- Wrong data types

Solution 1: tool_use with Schema

Define a tool that represents your desired output structure:

```
{
  "name": "extract_invoice_data",
  "description": "Extract structured data from an invoice document",
  "input_schema": {
    "type": "object",
    "properties": {
      "vendor_name": {
        "type": "string",
        "description": "The company or person who issued the invoice"
      },
      "invoice_number": {
        "type": "string",
        "description": "The unique invoice identifier"
      },
      "line_items": {
        "type": "array",
        "items": {
          "type": "object",
          "properties": {
            "description": { "type": "string" },
            "quantity": { "type": "number" },
            "unit_price": { "type": "number" },
            "total": { "type": "number" }
          },
          "required": ["description", "quantity", "unit_price", "total"]
        }
      },
      "total_amount": { "type": "number" },
      "currency": {
        "type": "string",
        "enum": ["USD", "EUR", "GBP", "other"]
      }
    },
    "required": ["vendor_name", "invoice_number", "line_items", "total_amount", "currency"]
  }
}
```

Use `tool_choice: {"type": "tool", "name": "extract_invoice_data"}` to force Claude to call this tool, guaranteeing structured output – on Sonnet 5, Opus 5, and the 4.x family. **Fable 5.1 / Mythos 5.1 and Opus 5.5 reject forced tool use with a 400** (Domain 2 §2.3); there, prefer Solution 3 below, or `auto` plus an instruction naming the tool with `strict: true`.

Solution 2: Strict Mode

Add `"strict": true` to the tool definition for guaranteed schema compliance:

```
{
  "name": "extract_invoice_data",
  "strict": true,
  "input_schema": {
    "type": "object",
    "properties": { ... },
    "required": ["vendor_name", "invoice_number", "line_items", "total_amount", "currency"],
    "additionalProperties": false
  }
}
```

Strict mode requirements:

- `additionalProperties: false` must be set
- ALL properties must be listed in `required`

Solution 3: output_config (Direct JSON Output)

Instead of `tool_use`, you can request JSON output directly. The parameter is `output_config.format`. The older top-level `output_format` is **deprecated** – don't use it in new code – and the previous `structured-outputs-2025-11-13` beta header is no longer required:

```
response = client.messages.create(
    model="claude-opus-5-5",
    messages=[{"role": "user", "content": "Extract data from this invoice: ..."}],
    output_config={
        "format": {
            "type": "json_schema",
            "schema": {
                "type": "object",
                "properties": {
                    "vendor_name": {"type": "string"},
                    "total_amount": {"type": "number"}
                },
                "required": ["vendor_name", "total_amount"],
                "additionalProperties": false
            }
        }
    }
)
```

SDK helper: `client.messages.parse()` (Python/TypeScript) wraps this pattern – pass a Pydantic model or Zod schema and it sends `output_config.format`, validates the response against your schema, and returns the parsed object via `response.parsed_output`. Prefer it over hand-parsing JSON when using structured outputs.

Schema Compliance vs Semantic Correctness

Important exam concept: Structured output guarantees **schema compliance** (correct types, required fields present, valid enums) but does NOT guarantee **semantic correctness**.

Schema compliance:  `"total_amount": 150.00` (correct type, field exists) Semantic error:  `"total_amount": 150.00` when line items sum to 175.00

You still need validation logic to catch semantic errors.

Schema Design Patterns

Required vs Optional Fields:

```
{
  "purchase_order_number": {
    "type": ["string", "null"],
    "description": "PO number if present on the invoice, null if not found"
  }
}
```

Use nullable fields when source documents may not contain the information. Making everything required can force Claude to fabricate data.

Enum with "other":

```

{
  "document_type": {
    "type": "string",
    "enum": ["invoice", "receipt", "contract", "purchase_order", "other"]
  },
  "document_type_detail": {
    "type": ["string", "null"],
    "description": "If document_type is 'other', describe the document type"
  }
}

```

Enum with "unclear":

```

{
  "payment_status": {
    "type": "string",
    "enum": ["paid", "unpaid", "partial", "unclear"]
  }
}

```

Adding `"unclear"` as an option prevents Claude from guessing when the information is ambiguous.

JSON Schema Limitations in Strict Mode

Strict mode supports a subset of JSON Schema:

Supported	NOT Supported
<code>type</code> (object, array, string, integer, number, boolean, null)	<code>minimum</code> , <code>maximum</code> , <code>multipleOf</code>
<code>properties</code> , <code>required</code>	<code>minLength</code> , <code>maxLength</code>
<code>enum</code> , <code>const</code>	<code>pattern</code> (regex), <code>patternProperties</code>
<code>items</code> (for arrays)	<code>oneOf</code> , <code>if/then/else</code>
<code>anyOf</code> , <code>allOf</code> , <code>\$ref/\$defs</code>	Recursive schemas
<code>additionalProperties: false</code> (required on every object)	<code>additionalProperties</code> set to anything other than <code>false</code>
<code>format</code> (<code>date-time</code> , <code>time</code> , <code>date</code> , <code>duration</code> , <code>email</code> , <code>hostname</code> , <code>uri</code> , <code>ipv4</code> , <code>ipv6</code> , <code>uuid</code>)	Complex array constraints

Earlier versions of structured outputs did not support `anyOf`/`allOf` – they do now. The Python and TypeScript SDKs strip unsupported constraints from the schema they send and validate them client-side instead.

Implication: Numeric range validation, string length validation, regex patterns, and complex conditional schemas must be handled in your validation code, not in the schema.

4.4 Validation-Retry Loops

The Pattern

```
max_retries = 3

for iteration in range(max_retries):
    response = extract_data(document, prompt)
    errors = validate(response)

    if not errors:
        return response # Success

    # Append specific errors to prompt for next attempt
    prompt += f"""
    Your previous extraction had these validation errors:
    {json.dumps(errors, indent=2)}

    Please correct these specific issues and try again.
    """

raise ExtractionError(f"Failed after {max_retries} attempts: {errors}")
```

What Validation Catches

Schema validation (can catch reliably):

- Missing required fields
- Wrong data types
- Invalid enum values
- Malformed dates or emails

Semantic validation (needs custom logic):

- Line items don't sum to total
- Dates are in the future when they shouldn't be
- Referenced entities don't exist
- Cross-field contradictions

Self-Correction Patterns

Calculated vs Stated:

```
{
  "stated_total": 150.00,
  "calculated_total": 175.00,
  "conflict_detected": true,
  "conflict_note": "Stated total ($150) doesn't match sum of line items ($175)"
}
```

By extracting both the stated value and a calculated value, you can detect discrepancies automatically.

When Retries Don't Work

Retries are ineffective when the information is **absent from the source document**. If the invoice doesn't have a PO number, retrying won't make one appear.

Solution: Use nullable/optional fields and accept null for missing data rather than retrying.

Tracking False Positives

Add a `detected_pattern` field to track which code constructs trigger findings:

```
{
  "issue": "Potential SQL injection",
  "detected_pattern": "string concatenation in query",
  "file": "users.py",
  "line": 42
}
```

This allows analysis of dismissal patterns – if developers consistently dismiss "string concatenation in query" findings, the prompt criteria for that pattern may need refinement.

4.5 Batch Processing

Message Batches API

Feature	Detail
Cost	50% savings vs synchronous
Latency	Up to 24 hours, no SLA
Correlation	<code>custom_id</code> per request – results come back in any order , so key by <code>custom_id</code> , never by position
Multi-turn	NOT supported in a single batch request
Max output	128k normally; up to 300k with beta header <code>output-300k-2026-03-24</code> on Opus 5.5 / 5 / 4.8 / 4.7 / 4.6 and Sonnet 5 / 4.6 (not Fable 5.1)
Result states	<code>succeeded</code> , <code>errored</code> , <code>canceled</code> , <code>expired</code> – poll <code>batches.retrieve(id).processing_status</code> until <code>"ended"</code> , then stream <code>batches.results(id)</code>

Creating a Batch

```
batch = client.messages.batches.create(
  requests=[
    {
      "custom_id": "invoice-001",
      "params": {
        "model": "claude-sonnet-5",
        "max_tokens": 4096,
        "messages": [
          {"role": "user", "content": f"Extract data from: {invoice_001_text}"}
        ],
        "tools": [extract_tool],
        "tool_choice": {"type": "tool", "name": "extract_invoice_data"} # forced: Sonnet 5 / Opus 5 / 4.x only -
      }
    },
    {
      "custom_id": "invoice-002",
      "params": { ... }
    }
  ]
)
```

When to Use Batch vs Synchronous

Synchronous	Batch
Pre-merge code review checks	Overnight code analysis
Real-time user interactions	Weekly audit reports
Blocking CI steps	Nightly test generation
Interactive debugging	Bulk document processing

Failure Handling

Don't resubmit the entire batch when some requests fail. Use `custom_id` to identify and resubmit only failures:

```
failed = [r for r in results if r.status == "failed"]
resubmit = [orig for orig in original if orig.custom id in [f.custom id for f in failed]]
```

Pre-Batch Refinement

Always refine your prompts on a small sample before batch-processing large volumes. This catches prompt issues before you waste credits on 10,000 documents.

4.6 Multi-Pass Review

The Self-Review Problem

When Claude generates code and then reviews it in the same session, it retains the reasoning context from generation. It "remembers" why it made each decision, making it less likely to question those decisions.

This is called **reasoning context bias**.

The Solution: Independent Review Instances

```

Session 1: Claude generates code
      ↓
      (completely separate session)
      ↓
Session 2: Claude reviews the code
            (no knowledge of why decisions were made)

```

Multi-Pass Architecture for Large PRs

Pass 1 – Per-file local analysis:

- Review each file independently
- Consistent depth across all files
- Catches: bugs, style issues, type errors, security vulnerabilities
- Separate prompt for each file

Pass 2 – Cross-file integration:

- Review how files interact with each other
- Different prompt focused on integration concerns
- Catches: data flow issues, interface mismatches, missing error handling at boundaries

- Receives results from Pass 1 for context

Why Not Single-Pass?

For large PRs (10+ files), single-pass review suffers from:

1. **Attention dilution** – Model focuses on early files, gives less attention to later ones
2. **Inconsistent standards** – Different quality of review across files
3. **Missing integration issues** – Can't see cross-file problems when reviewing one file at a time in a single pass

4.7 Thinking, Effort, and the Current Request Surface

The request shape for prompting Claude changed substantially across the 4.6 → 5 generations. Several parameters that older material treats as standard now return **400** on current models.

Adaptive Thinking Replaced Thinking Budgets

```
response = client.messages.create(
    model="claude-opus-5-5",
    max_tokens=16000,
    thinking={"type": "adaptive", "display": "summarized"},
    output_config={"effort": "high"},
    messages=[...],
)
```

Model	Thinking config	Omitting <code>thinking</code>	<code>budget_tokens</code>	<code>temperature</code> / <code>top_p</code> / <code>top_k</code>
Claude Fable 5.1 / Mythos 5.1	<code>{"type": "adaptive"}</code> or omit; <code>disabled</code> → 400	Runs adaptive (always on)	400	400
Claude Fable 5	<code>{"type": "adaptive"}</code> or omit	Runs adaptive (always on)	400	400
Claude Opus 5.5	<code>{"type": "adaptive"}</code> or omit; <code>disabled</code> → 400 at every effort	Runs adaptive (always on); default effort <code>medium</code>	400	400
Claude Opus 5	<code>{"type": "adaptive"}</code> or omit; <code>disabled</code> accepted only at effort <code>high</code> or below	Runs adaptive by default	400	400
Claude Opus 4.8 / 4.7	<code>{"type": "adaptive"}</code>	Runs without thinking	400	400
Claude Sonnet 5	<code>{"type": "adaptive"}</code>	Runs adaptive	400	400
Claude Haiku 4.5	<code>{"type": "enabled", "budget_tokens": N}</code>	No thinking	Required for thinking	Allowed

The subtlety worth flagging: on **Opus 4.8/4.7** you must set `adaptive` explicitly or you get no thinking at all, on **Opus 5** thinking is on by default, and on **Opus 5.5** it can't be turned off at all – a `disabled` setting carried forward from 4.8 or Opus 5 is a 400; lower `effort` instead. Prefer that anyway: with thinking off, Opus 5 sometimes writes a tool call into its visible text instead of a `tool_use` block, so the call never runs and no error is raised.

Effort

`effort` is GA (no beta header) and lives **inside** `output_config`, not at the top level:

```
output_config={"effort": "xhigh"} # low | medium | high | xhigh | max
```

The API default is `high` on every model that supports effort **except Opus 5.5, which defaults to `medium`** – one level below Opus 5, so a request that omits `effort` runs cheaper and shallower after a migration; set it explicitly and re-run your sweep (Opus 5.5 also thinks more per turn than Opus 5 at the same level, most of all at `xhigh` / `max`, so leave room in `max_tokens`). Setting effort to the model's default is identical to omitting it. Haiku 4.5 rejects `effort`, and `xhigh` is not available on Opus 4.6 / Sonnet 4.6. Claude Code sets effort **per model** (`/effort`), so a level saved for one model doesn't carry to a newly released one, and `maxEffortLevel` caps it org-wide. Anthropic's per-model guidance: on **Fable 5.1, Opus 5, and Sonnet 5 start at `high` and sweep** – step up to `xhigh` / `max` only where evals show headroom, and use `low` / `medium` liberally for routine or latency-sensitive routes (lower effort on these models often beats `xhigh` on the previous generation); on **Opus 5.5 start at `medium` and sweep in both directions**; on **Opus 4.8 / 4.7 start at `xhigh`** for coding and agentic work. Lower effort means fewer, more-consolidated tool calls and terser output. Effort matters more on these models than on any prior generation, so **re-tune it when you migrate**, don't carry the old value across. Changing the top-level `effort` mid-conversation invalidates the message cache; on Fable 5.1, Mythos 5.1, Opus 5.5, and Opus 5, a **per-message effort** change – a `role: "system"` message with empty `content` carrying `output_config.effort` (beta `mid-conversation-output-config-2026-07-01`, Claude API and Google Cloud) – keeps the prefix intact and, having no text, can sit anywhere in `messages`.

Thinking Display

`display: "omitted"` is now the **default** on Fable 5 / 5.1, Opus 5.5, Opus 5, Opus 4.8/4.7, and Sonnet 5 – a silent change from Opus 4.6 and Sonnet 4.6, where it was `"summarized"`. Thinking still happens and is still billed identically; only visibility changes. If you stream reasoning to users, the default looks like a long pause before any output, so set `display: "summarized"` explicitly. The raw chain of thought is never exposed on any model. On Fable 5.x and Opus 5.5 the text Claude writes *between tool calls* comes back as progress-update `thinking` blocks rather than `text` blocks, so under the default it is empty – a UI that streamed those notes on Opus 5 goes quiet between tool calls, with no error. `display: "updates"` (beta `thinking-display-updates-2026-08-18`) returns those progress notes as readable text while the reasoning itself stays hidden; `"summarized"` returns both.

Assistant Prefill Is Gone

Prefilling the last assistant turn to force a response format returns **400** on Fable 5 / 5.1, Opus 5.5, Opus 5, Sonnet 5, and the whole 4.6/4.7/4.8 family. Use structured outputs (`output_config.format`) or system-prompt instructions instead. This retires a prompt-engineering technique that a lot of older material still recommends.

Mid-Conversation System Messages

On Claude Opus 5.5, Opus 5, Opus 4.8, Fable 5 / 5.1, and Mythos 5 / 5.1 (not Sonnet 5, no beta header) you can append `{"role": "system", "content": "..."} to the messages array rather than editing the top-level system field. This is the prompt-injection-safe operator channel, and – because it doesn't touch the cached prefix – the cheap one. Constraints: it must follow a user message (or an assistant message ending in server-tool use), can't be messages[0], and must either be last or be followed by an assistant turn.`

Two extensions arrived with Fable 5.1 (both beta): **turn-scoped** messages with `clear_at: "next_user_message"` (header `mid-conversation-system-clear-at-2026-08-21`) render for one turn, then stay in the transcript cleared – the right shape for a per-turn reminder such as "batch independent tool calls", which Fable 5.1 needs more often than Fable 5 in long loops; and **tool changes** via `tool_addition` / `tool_removal` blocks (header `mid-conversation-tool-changes-2026-07-01`) – since 2026-09-22 a `tool_addition` can also carry a full definition (`inline-tools-2026-09-15`), so a schema change or a server-tool version bump no longer touches `tools` – both covered in Domain 2 §2.6. Opus 5.5 supports all of these. Never delete earlier copies of these messages: on Fable 5.1 and Opus 5.5 that counts as editing history.

Preserved Thinking: The History Is Now Part of the Contract

Every `thinking` block Fable 5.1 or Opus 5.5 emits records the model that produced it *and* is valid only against the exact `system`, `tools`, and message prefix that preceded it. Two rules follow:

1. **Model binding.** Fable 5.1 and Mythos 5.1 read each other's blocks and every earlier model's – Opus 5.5, Opus 5, Fable 5, Mythos 5, Sonnet, Haiku. Opus 5.5 reads Opus 5 and earlier Opus / Sonnet / Haiku blocks, but not Fable or Mythos ones. No older model reads Fable 5.1's or Opus 5.5's. So Opus 5 → Opus 5.5 → Fable 5.1 keeps its reasoning at every hop; any other switch (a fallback from Opus 5.5 to Opus 5, a router from Fable 5.1 to Opus 5.5) silently drops the unreadable blocks (unbilled) and the target model re-plans.
2. **Prefix binding.** Editing, reordering, or removing earlier turns invalidates every later thinking block. The API enforces the check on Fable 5.1 and Opus 5.5 by default for accounts created on or after 2026-08-31 (a replayed block after such an edit is a 400); older accounts are checked only when they set `thinking.block_binding.prefix_mismatch_behavior` (`"error"` or `"drop_block"`). Append-only histories, server-side context editing, and API-written compaction are all safe, because the check compares the conversation *as you sent it* – a keep-tail summary you wrote yourself is not (Domain 5 §5.7).

If Claude Code, claude.ai, Managed Agents, or the Agent SDK own your history, they already comply. If your code builds `messages` itself, run the three-step check from the migration guide: capture consecutive request bodies, diff `system` / `tools` / shared `messages` prefix, and fix every non-append change. Under the `thinking-binding-controls-2026-08-01` header every response carries `input_transformations`: `thinking_dropped` entries name blocks the API removed (`prefix_binding_mismatch` or `model_binding_mismatch`), and – on requests where the check isn't enforced, i.e. older accounts – `thinking_mismatch_allowed` entries (since 2026-09-14) name blocks that *would* have failed but still reached the model. Log the second kind now to find history edits in production before you opt into enforcement; `/claude-api/preserved-thinking-migration` in Claude Code automates the capture-diff-replay loop. Claude Code's own September releases fixed several cases where `--continue` / `--resume` re-sent earlier messages in a changed form – the same check biting Anthropic's harness.

4.8 Prompt Caching

Caching is a prompt-*structure* problem, which is why it belongs here rather than in an ops runbook.

Prefix match: the cache keys on an exact prefix. Any byte change anywhere in the prefix invalidates everything after it. Render order is `tools` → `system` → `messages`, so a tool list that reorders between requests invalidates the system prompt and the whole conversation behind it.

Design rule: stable content first (frozen system prompt, deterministically ordered tool list), volatile content (timestamps, per-request IDs, the actual question) after the last `cache_control` breakpoint.

Mechanic	Detail
Breakpoints	Max 4 per request (<code>cache_control: {"type": "ephemeral"}</code>)
Minimum prefix	Model-dependent – 512 tokens on Fable 5.x / Mythos 5.x / Opus 5.5 / Opus 5; 1,024 on Opus 4.8, Sonnet 5, Sonnet 4.6/4.5; 2,048 on Opus 4.7; 4,096 on Opus 4.6/4.5 and Haiku 4.5. Shorter prefixes silently don't cache
Pricing	Cache reads cost 10% of the input price (2.5% on Fable 5.1 / Mythos 5.1 – \$0.25/MTok; 5% on Opus 5.5 – \$0.20/MTok); 5-minute writes 1.25×, 1-hour writes 2×. On Opus 5.5 a cached turn costs a twentieth of an uncached one – the strongest cost argument yet for a stable prefix
Verification	<code>usage.cache_read_input_tokens</code> . Zero across repeated requests means something is invalidating the prefix. Cache diagnostics is GA on the Claude API since 2026-09-23 (no beta header): send <code>diagnostics: {"previous_message_id": null}</code> on the first turn and the previous response <code>id</code> after that, and <code>response.diagnostics</code> reports exactly where consecutive requests diverged. The field is now always present (<code>null</code> when you didn't opt in), and a fingerprint is stored only for requests that include the <code>diagnostics</code> object – include it on every turn or a later one reports <code>previous_message_not_found</code>
Pre-warming	<code>max_tokens: 0</code> warms a cache entry without generating – with the same thinking and <code>effort</code> configuration your real traffic uses, or the entry is never hit

Silent invalidators to audit for: `datetime.now()` in the system prompt, JSON serialized with non-deterministic key order, a tool set that varies per request, a session ID or user name interpolated into the prefix, switching `speed` (fast mode) mid-conversation, changing the top-level `effort` or thinking configuration between requests (use per-message effort on Fable 5.1 / Opus 5.5 / Opus 5), and changing a task-budget value (it is rendered into the prompt).

Interaction with tool search: deferred tools are excluded from the system-prompt prefix and discovered tools are appended inline, so `defer_loading` preserves the cache. A tool with `defer_loading: true` cannot also carry `cache_control` – put the breakpoint on a non-deferred tool.

Domain 4 Practice Questions

Q1: A code review system flags "use your best judgment" as a criterion for reporting issues. What should be changed?

- A) Add more examples of past reviews
- B) Replace with explicit criteria defining exactly what qualifies as a reportable issue
- C) Increase the model's temperature for more nuanced judgment
- D) Add a confidence score threshold

Answer: B – Vague instructions like "use your best judgment" produce inconsistent results. Replace with explicit criteria that define exactly what qualifies and what doesn't.

Q2: An invoice extraction system uses strict mode but sometimes returns a total that doesn't match the sum of line items. What's happening?

- A) Strict mode is broken
- B) The schema is misconfigured
- C) Strict mode guarantees schema compliance, not semantic correctness
- D) The model needs fine-tuning

Answer: C – Strict mode ensures the output conforms to the JSON schema (correct types, required fields) but cannot validate semantic correctness (whether the math adds up). Custom validation logic is needed for that.

Q3: A batch processing job has 1,000 documents. 50 fail. What's the correct approach?

- A) Resubmit the entire batch of 1,000
- B) Use `custom_id` to identify and resubmit only the 50 failed documents
- C) Switch to synchronous processing
- D) Increase the batch timeout

Answer: B – Use `custom_id` to correlate requests with responses and resubmit only the failures. Resubmitting the entire batch wastes credits and reprocesses already-successful documents.

Q4: An extraction service written for an older model sets `thinking={"type": "enabled", "budget_tokens": 8000}` and `temperature=0`. It is being moved to Claude Opus 5. What happens?

- A) It works unchanged; both parameters are still supported
- B) Both parameters return a 400 – use `thinking={"type": "adaptive"}` and control depth with `output_config.effort`
- C) `budget_tokens` is ignored silently and `temperature` still applies
- D) It works, but thinking is disabled

Answer: B – `budget_tokens` and the sampling parameters (`temperature` / `top_p` / `top_k`) are removed on Fable 5 / 5.1, Opus 5.5, Opus 5, Sonnet 5, and the 4.7/4.8 family, and return a 400. Adaptive thinking plus `effort` replaces the fixed-thinking-budget concept. Assistant prefills also 400 on these models.

Domain 5: Context Management & Reliability (15%)

Covers conversation context management, escalation patterns, error propagation, large codebase context strategies, human review workflows, and information provenance.

5.1 Conversation Context Management

The Context Window Is Finite

Claude has a limited context window. Every message, tool result, and system prompt consumes tokens. Long conversations degrade as earlier information gets compressed or lost.

Current sizes, because "1M tokens" changes the shape of the problem without eliminating it:

Model	Context window	Max output
Claude Fable 5.1 / Fable 5 / Opus 5.5 / Opus 5 / Sonnet 5	1M tokens	128k
Claude Opus 4.8 / 4.7 / 4.6, Sonnet 4.6	1M tokens	128k
Claude Haiku 4.5	200k tokens	64k

Two traps that come with a 1M window:

1. **A bigger window is not free.** Every token in it is billed on every turn, and the "lost in the middle" effect below gets *worse* with length, not better. Filling a 1M window because you can is a cost and a quality regression.
2. **Token counts changed.** Opus 4.7 introduced a new tokenizer, carried by Opus 4.8, Opus 5, Opus 5.5, Sonnet 5, and Fable 5 / 5.1: the same text produces roughly 30% more tokens than on pre-4.7 models (1M tokens $\approx$ 555k words now, versus $\approx$ 750k before). Any budget, chunk size, or threshold calibrated on an older model needs re-baselining with `messages.count_tokens` – never with `tiktoken`.

Progressive Summarization Risks

When context is compressed (via `/compact` or automatic compression), the system summarizes earlier conversation. This introduces risks:

What gets lost in summarization:

- Exact numerical values (`$149.99` becomes "approximately \$150")
- Specific dates (`2024-01-15` becomes "mid-January")
- Precise percentages (`23.7%` becomes "about 24%")
- Order numbers, customer IDs, reference codes
- Exact error messages and stack traces

Solution: Case Facts Block

Extract critical transactional data into a persistent structured block that survives summarization:

CASE FACTS (do not summarize):

- Customer ID: #12345
- Order: #67890
- Order date: 2024-01-15
- Total: \$149.99
- Issue: Damaged product on arrival
- Refund requested: Full refund (\$149.99)

The "Lost in the Middle" Effect

Models process the **beginning** and **end** of long inputs more reliably than the **middle**. Information placed in the middle of a large context is more likely to be overlooked.

Mitigation strategies:

- 1. Place key findings summaries at the **beginning** of aggregated inputs
- 2. Use explicit section headers to create navigation structure
- 3. Keep critical information out of the middle of long documents
- 4. If you must include lots of content, bookend the important parts

Tool Output Trimming

Tool results can be verbose. A customer order lookup might return 40 fields when you only need 5.

Bad: Return all 40 fields, consuming context for nothing.

Good: Trim to relevant fields before returning to Claude:

```
{
  "order_id": "67890",
  "status": "delivered",
  "delivery_date": "2024-01-20",
  "total": 149.99,
  "items": ["Widget Pro (x2)"]
}
```

What Survives /compact

Survives	May Be Lost
CLAUDE.md (re-read from disk)	Earlier conversation details
System prompts	Specific tool results
Recent messages	Nuanced reasoning from earlier turns
Auto-memory (MEMORY.md)	Exact quotes and numbers from early context

5.2 Escalation Patterns

When to Escalate Immediately

Trigger	Why	Action
Customer explicitly requests a human	Respecting user autonomy	Escalate immediately, no further investigation
Policy gap or exception	Agent can't make policy decisions	Escalate with structured context
No progress after reasonable attempts	Agent is stuck	Escalate with summary of what was tried

When NOT to Escalate

Bad Trigger	Why It's Wrong
High sentiment/emotion	Sentiment ≠ complexity. An angry customer may have a simple issue.
Self-reported low confidence	Claude's confidence scores are poorly calibrated. A "low confidence" response might be correct, while a "high confidence" response might be wrong.
Long conversation	Length doesn't indicate need for escalation. Some issues legitimately take multiple turns.

The Sentiment ≠ Complexity Distinction

This is a specifically tested concept:

"I'M SO FRUSTRATED WITH THIS!!!" + simple billing error
→ Agent can resolve. Don't escalate based on caps and exclamation marks.

"I have a question about the interaction between your enterprise SLA and the new EU data residency requirements" + calm tone
→ May need escalation due to policy complexity, despite calm sentiment.

Multiple Customer Matches

When a customer lookup returns multiple matches:

Wrong: Pick the most likely match using heuristics. **Right:** Ask the customer for additional identifying information to disambiguate.

"I found multiple accounts matching that name. Could you provide your email address or the last four digits of your phone number so I can locate the right account?"

Structured Escalation Handoff

When escalating, provide a structured summary because the human agent typically can't see the conversation transcript:

ESCALATION SUMMARY
=====

Customer: Jane Doe (#12345)
Contact: jane@example.com | 555-0123
Issue type: Refund request – damaged product
Order: #67890 (\$149.99, delivered 2024-01-20)

Actions taken by agent:

1. Verified customer identity
2. Confirmed order and delivery details
3. Customer provided photos of damage
4. Refund amount (\$149.99) exceeds automated approval limit (\$100)

Recommended action: Manual approval for full refund + replacement
Urgency: Standard (customer is frustrated but not at risk of churn)

5.3 Error Propagation

Structured Error Context

When errors occur, propagate them with full context:

```
{
  "failure_type": "timeout",
  "service": "knowledge_base_search",
  "attempted_query": "return policy for electronics over $500",
  "timeout_duration_ms": 30000,
  "partial_results": null,
  "alternative_approaches": [
    "Try searching with simpler terms",
    "Check the FAQ section directly",
    "Ask the customer for the specific product"
  ],
  "is_retryable": true,
  "retry_after_ms": 5000
}
```

This gives Claude enough information to make an intelligent recovery decision.

Anti-Patterns

1. Generic error messages:

```
{ "error": "Operation failed" }
```

Claude can't recover intelligently because it doesn't know what failed or why.

2. Silent suppression:

```
{ "results": [] } // Returned when the search engine was actually down
```

Claude will confidently tell the user "I couldn't find any information about that" when the truth is "I wasn't able to search." These are fundamentally different situations.

3. Workflow termination:

```
Step 1: Get customer info → Success
Step 2: Search knowledge base → Timeout
Step 3: ABORT ENTIRE WORKFLOW
```

Better: Continue with partial results, note the gap, and offer alternatives.

Access Failure vs Empty Result

Situation	HTTP Status	Meaning	Agent Response
Valid search, no matches	200 + empty array	No relevant content exists	"I didn't find any matching articles"
Search service down	503 / timeout	Search didn't execute	"I wasn't able to search right now, let me try another approach"

Subagent Error Recovery

Subagents should implement **local recovery** for transient failures:

```
Subagent: Search for "return policy"
→ Timeout
→ Retry with backoff (local recovery)
→ Success on second attempt
→ Return results to coordinator
```

Only propagate errors the subagent cannot resolve. When propagating, include:

- What was attempted
- What partial results were obtained
- What the failure was
- Whether it's retryable

5.4 Large Codebase Context Management

Context Degradation in Extended Sessions

In long sessions, Claude's reliability degrades:

- References "typical patterns" instead of specific code found earlier
- Gives inconsistent answers about the same code
- Forgets earlier discoveries
- Starts making assumptions instead of checking

Mitigation Strategies

1. Scratchpad Files

Persist findings to files that survive context compression:

```
# scratchpad.md
## Auth System Findings
- Entry point: src/auth/middleware.ts:23
- Uses JWT with RS256
- Token expiry: 1 hour
- Refresh token: 7 days
- Role check: src/auth/roles.ts:45
- Known issue: No token revocation mechanism
```

When context is compressed, Claude can re-read the scratchpad to recover findings.

2. Subagent Delegation

Isolate verbose exploration in subagents:

```
Main context: Clean, focused on the task
↓
Explore subagent: Reads 50 files, searches codebase extensively
↓
Returns: Concise summary of findings
↓
Main context: Receives only the summary, stays clean
```

3. `/compact` Proactive Use

During extended sessions, proactively compact context before it degrades naturally. Better to compact with a structured summary than to let automatic compression lose information unpredictably.

4. Crash Recovery Pattern

For long-running multi-agent workflows:

```
# Each agent exports state to a known location
agent.save_state("/tmp/workflow/agent-1-state.json")

# Coordinator maintains a manifest
manifest = {
  "agent-1": {"status": "complete", "state": "/tmp/workflow/agent-1-state.json"},
  "agent-2": {"status": "in_progress", "state": "/tmp/workflow/agent-2-state.json"},
  "agent-3": {"status": "pending"}
}

# On resume, coordinator loads manifest and picks up where it left off
```

5.5 Human Review Workflows

Stratified Random Sampling

Don't just sample randomly from all outputs. Stratify by risk or category:

```
Category A (high-risk): Sample 20% of outputs
Category B (medium-risk): Sample 10% of outputs
Category C (low-risk): Sample 5% of outputs
```

This concentrates human review effort on areas most likely to have errors.

Field-Level Confidence Scores

Instead of a single confidence score for the entire extraction, provide per-field confidence:

```
{
  "vendor_name": { "value": "Acme Corp", "confidence": 0.99 },
  "total_amount": { "value": 149.99, "confidence": 0.95 },
  "purchase_order": { "value": "PO-2024-001", "confidence": 0.72 },
  "tax_id": { "value": "12-3456789", "confidence": 0.45 }
}
```

Calibration: Confidence scores must be calibrated against labeled validation sets. A 0.9 confidence should mean the field is correct ~90% of the time. Without calibration, confidence scores are meaningless.

Accuracy by Document Type

The hidden failure pattern:

```
Overall accuracy: 97% ← Looks great!

Breakdown:
  Standard invoices: 99.5% ← Excellent
  Handwritten notes: 45% ← Terrible
  Foreign language docs: 62% ← Bad
```

Aggregate metrics mask poor performance on specific document types. Always break down accuracy by type and field.

Per-Document-Type Reporting

Report accuracy metrics for each document type AND each field within that type:

Standard Invoices:

vendor_name: 99.8%
total_amount: 99.2%
line_items: 97.5%
tax_id: 94.1%

Handwritten Notes:

vendor_name: 85.0%
total_amount: 42.0% ← Critical failure
line_items: 38.0% ← Critical failure

This reveals where the system actually needs improvement rather than hiding behind an aggregate number.

5.6 Information Provenance

Claim-Source Mappings

When subagents produce findings, they should output structured provenance:

```
{
  "claim": "Revenue increased 15% year-over-year in Q4 2024",
  "evidence": "Q4 earnings report, page 3, paragraph 2",
  "source_url": "https://example.com/earnings/q4-2024",
  "document_name": "Q4 2024 Earnings Report",
  "publication_date": "2025-01-15",
  "source_type": "primary"
}
```

Handling Conflicting Data

When sources disagree, **annotate the conflict** rather than silently picking one:

```
{
  "field": "Q4 revenue",
  "values": [
    {
      "value": "$2.3B",
      "source": "Company press release (2025-01-15)",
      "note": "Preliminary figures"
    },
    {
      "value": "$2.28B",
      "source": "SEC filing (2025-02-28)",
      "note": "Audited figures"
    }
  ],
  "resolution": "SEC filing is the authoritative source (audited)",
  "conflict_type": "minor_discrepancy"
}
```

Anti-pattern: Silently selecting one value without noting the disagreement. This hides potential data quality issues.

Temporal Data

Include publication and collection dates to prevent temporal confusion:

```
{
  "claim": "Unemployment rate is 3.7%",
  "data_as_of": "2024-11-01",
  "source_published": "2024-12-06",
  "retrieved_on": "2025-01-10"
}
```

Without dates, a reader might think two different unemployment figures from different months represent a contradiction when they're actually both correct for their respective time periods.

Synthesis Output Quality

When producing research synthesis:

1. Distinguish certainty levels:

- "Well-established: Revenue grew 15% (confirmed by SEC filing and two analyst reports)"
- "Contested: Market share estimates range from 12% to 18% depending on methodology"
- "Single-source: Only one report mentions the planned acquisition"

2. Preserve source characterization:

- Don't merge or reinterpret what sources say
- Quote or closely paraphrase the original claim
- Note the source's own caveats and qualifications

3. Render appropriately by content type:

- Financial data → tables with source attribution
- News/events → chronological prose
- Technical specifications → structured lists
- Disputed claims → side-by-side comparison

5.7 Server-Side Context Management

Sections 5.1 and 5.4 cover context management as something *you* do – case-facts blocks, scratchpads, trimming, delegating to subagents. The API now offers three server-side mechanisms that do part of this work for you. They are complementary, not alternatives, and knowing which one solves which problem is the point.

Mechanism	What it does	Where state goes
Context editing	<i>Clears</i> old tool results or thinking blocks before the model sees them	Deleted – gone
Compaction	<i>Summarizes</i> earlier context – on demand (you send a <code>compaction</code> request) or when input reaches a token threshold	Replaced by a signed summary block in the conversation
Memory tool	Lets Claude read and write persistent files	Outside the conversation, survives everything

Context Editing (beta `context-management-2025-06-27`)

Clears, does not summarize.

```

client.beta.messages.create(
    model="claude-opus-5-5",
    betas=["context-management-2025-06-27"],
    context_management={"edits": [
        {"type": "clear_thinking_20251015",
         "keep": {"type": "thinking_turns", "value": 2}},
        {"type": "clear_tool_uses_20250919",
         "trigger": {"type": "input_tokens", "value": 30000},
         "keep": {"type": "tool_uses", "value": 3},
         "clear_at_least": {"type": "input_tokens", "value": 5000},
         "exclude_tools": ["web_search"]}],
    },
    tools=[...], messages=[...],
)

```

- `clear_tool_uses_20250919` – clears old tool results (`clear_tool_inputs: true` also clears the parameters). Default trigger: 100,000 input tokens; default `keep`: 3.
- `clear_thinking_20251015` – clears thinking blocks. When combining both, `clear_thinking` **must be listed first**.
- `clear_at_least` exists to protect the prompt cache: clearing a trivial amount invalidates the cache for no benefit.
- The response reports what happened in `context_management.applied_edits`.
- On Fable 5.1 and Opus 5.5, server-side context editing **never invalidates thinking blocks**; deleting the same tool results client-side would (see below).

Compaction: On Demand (beta `compact-2026-09-04`) vs At a Threshold (beta `compact-2026-01-12`)

Summarizes rather than clears. Two kinds exist since 2026-09-14, and the docs say to **prefer on-demand compaction wherever it is available**:

	On demand	At a threshold
Who decides when	You, by sending a request	The API, when input tokens reach the trigger (default ~150k)
How	Send the conversation as it stands (same <code>system</code> and <code>tools</code>) with top-level <code>compaction: {"type": "summarize"}</code> ; the response is a single signed <code>compaction</code> block with <code>stop_reason: "compaction"</code> and no reply	<code>context_management.edits: [{"type": "compact_20260112"}]</code> on ordinary requests
What you send back	The block first in <code>messages</code> , in place of the messages it summarized, on every later request – exactly one block, unmodified, with the beta header	The whole <code>response.content</code> , appended as usual; the API drops what came before the block
Recent turns word for word	Yes – leave them out of the compaction request and send them after the block	Only by pausing after compaction and re-inserting them
Runs in the background	Yes – the conversation continues while the summary is written	No, it runs inside the request that hit the threshold
Kept turns keep their thinking (Fable 5.1 / Opus 5.5)	Yes, when the API wrote the summary and <code>system</code> / <code>tools</code> didn't change	No – strip or drop it
Availability	Fable 5.x, Mythos 5.x, Opus 5.5 / 5 / 4.8 / 4.7 / 4.6, Sonnet 5 / 4.6; Claude API, Claude Platform on AWS, Google Cloud, Foundry – not Bedrock	Same models

On-demand details worth carrying: `instructions` (up to 16,384 characters) *replaces* the default summarization prompt, so say what to retain and tell the model not to call tools; the summarization call is billed and reported under `usage.iterations` while the top-level token counts are zero; check `stop_reason` before looking for the block – `max_tokens`, `tool_use`, `refusal`, or `end_turn` mean no summary came back, and you simply continue and try again later; a request that ends in an unanswered tool call, or that carries `stop_sequences`, `output_config.format`, forced `tool_choice`, `context_management`, or a task budget's `remaining`, is rejected; and images, documents, and fetched URLs inside the summarized range are gone once the block replaces them. `role: "system"` messages in that range are summarized too – restate an instruction that still matters in a new `role: "system"` message right after your next `user` turn. The SDK tool runner does the swap for you via `compact_before_next_turn()`.

The critical integration detail for threshold compaction: append the whole `response.content` back to your `messages` on every turn, not just the extracted text. The compaction blocks in the response are what the API uses to replace compacted history on the next request. Pulling out the text string and appending that silently destroys the compaction state – and it fails quietly, which is the worst failure mode for something you only notice at turn 90.

On-demand compaction has two silent mistakes of its own: leaving summarized messages *after* the block sends them to Claude twice, and leaving the block out of a later request means Claude gets no summary at all. Summarized messages left *in front of* the block are the one case that errors (`compaction_block_misplaced`).

Client-side SDK compaction (`compaction_control` on the tool runner) is **deprecated** in favor of these.

Preserved Thinking: Compaction Shapes That Stay Valid

Fable 5.1 and Opus 5.5 add a constraint the other mechanisms didn't have: their thinking blocks are valid only against the exact history that preceded them, so **editing earlier turns invalidates every later block** (Domain 4 §4.7). Server-side context editing and compaction don't count as edits – the check compares the conversation *as you sent it*, and it accepts the swap when the API wrote the summary – which is now the strongest argument for moving trimming to the server. The shapes, and whether the thinking in kept turns survives:

Shape	Rule
Simple compaction	Replace the whole history with one summary (the API's block, or a summary message of your own) plus the new user turn; replay nothing else. No thinking blocks carry over, so nothing fails
Keep-tail compaction, API-written (on demand)	Kept turns keep valid thinking while three conditions hold: every compaction since a block was produced ran on a model with preserved thinking; the kept turns directly follow the summarized messages, unchanged, and start with a different role than the last summarized message (compacting exactly the <code>messages</code> of a request you already sent gets this right); and <code>system</code> plus the non-deferred <code>tools</code> are identical on the compaction request and every request after it
Keep-tail compaction, client-written	The kept assistant turns still carry thinking produced against the original turns, so those blocks fail. Strip their <code>thinking</code> / <code>redacted_thinking</code> blocks (text and tool calls can stay), or send <code>prefix_mismatch_behavior: "drop_block"</code>
Background compaction	With on-demand compaction, turns that arrived while the summary was written are kept turns and keep their thinking under the same conditions. With a client-written summary swapped in later, every block produced in between fails – send <code>"drop_block"</code> on each request that still carries pre-swap thinking, or compact synchronously

Nothing fails at compaction time; the failure comes on the first later request that replays the kept thinking where the check is enforced – a 400 by default, or dropped blocks under "drop_block", reported in input_transformations. Test it once: compact, keep one turn that holds a thinking block, send the next request with prefix_mismatch_behavior: "error", and expect a 200 with an empty input_transformations. What never works: snipping individual turns out of the middle of the transcript, deleting old tool results by hand, placing a role: "system" message between the block and the kept turns, or rebuilding system / tools between requests. To change system or tools safely, compact the whole conversation first (no kept turns), then change them on the next request; to change instructions or tools without touching either, append a role: "system" message (Domain 2 §2.6, Domain 4 §4.7). Dropping blocks once at a compaction boundary is cheap; invalidating them on every request restarts the prompt cache each time.

Memory Tool (memory_20250818)

A client-side tool – Anthropic defines the interface, you implement the file operations. Declared as {"type": "memory_20250818", "name": "memory"}, no input_schema.

Its role in this domain: it's the persistence layer that makes clearing safe. Combine it with context editing and Claude receives a warning to write anything important to memory before its tool results are cleared. That is the API-level version of the scratchpad pattern in §5.4.

Choosing

Situation	Mechanism
Agentic loop with dozens of verbose tool results	Context editing (clear_tool_uses)
Extended thinking plus a need to keep cache hits high	Context editing (clear_thinking)
Long research/analysis conversation that must keep its narrative	Compaction – on demand where available, threshold when the API should manage it inside ordinary requests
Application must pick the moment, keep the last turns word for word, or keep working while the summary is written	Compaction on demand
Findings that must survive across sessions, not just turns	Memory tool
Facts that must never be paraphrased	Case-facts block (§5.1) – no mechanism protects exact numbers better than restating them

5.8 Bounding and Recovering Long Runs

Task Budgets vs max_tokens

max_tokens is a ceiling the model can't see; hitting it truncates output mid-thought. A task budget (output_config.task_budget, beta task-budgets-2026-03-13, minimum 20,000) is a ceiling the model can see, so it paces itself and lands the work. Available on Claude Fable 5.1 / Mythos 5.1, Fable 5 / Mythos 5, Opus 5.5, Opus 5, and Opus 4.8/4.7 – not on Sonnet 5, Opus 4.6, or Haiku 4.5. The budget counts what Claude generates plus the tool results it reads this turn – not the full history you resend. Leave remaining unset in a normal loop; only pass it when you rewrite or compact history yourself and the server can no longer derive prior spend – and never on an on-demand compaction request or a request that carries its block (400). Two reliability footnotes: a budget that is obviously too small for the task

makes Claude decline, scope down, or stop early with a partial result (raise the budget before debugging anything else), and the budget value is rendered into the prompt, so changing it mid-task is a cache miss.

Managed Agents **session budgets** are a different thing: hard, dollar-denominated, platform-enforced caps on one session. A task budget is advisory and token-denominated.

Refusals Are a Reliability Concern, Not Just a Safety One

A `refusal` arrives as HTTP 200 with `stop_reason: "refusal"` and a `stop_details.category`. Code that reads `content` without checking `stop_reason` treats a refusal as a successful empty answer – the same class of bug as treating a failed search as zero results (§5.3). For production paths, enable server-side fallback (`betas=["server-side-fallback-2026-07-01"]`, `fallbacks="default"`) so the request is re-routed by category rather than dropped. See Domain 1 §1.1. Opus 5.5 widened the classifier set – a biology classifier joins the cybersecurity one, and prompts that push the model to reproduce its internal reasoning can be declined as `reasoning_extraction` – so refusals are now a first-class path on the default model, not just the flagship. Since 2026-09-24 a refusal before any output is **billed** when its category is `bio`, `frontier_llm`, or `reasoning_extraction`, and every refusal counts against rate limits, so budget for them. Mythos 5.1's safeguards depend on its Project Glasswing access program, so handle `refusal` there as well. And a fallback from Fable 5.1 or Opus 5.5 usually lands on a model that can't read their thinking blocks (only Fable 5.1 / Mythos 5.1 read Opus 5.5's), so budget for a re-planning turn.

Domain 5 Practice Questions

Q1: An agent is processing a customer support request. After context compression, the customer's order number (\$149.99 order #67890) was summarized as "a recent order." What should have been done to prevent this?

- A) Increase the context window size
- B) Extract critical transactional data into a persistent case facts block
- C) Disable context compression
- D) Repeat the order details in every message

Answer: B – Extracting transactional facts (amounts, order numbers, dates) into a structured block that persists through compression prevents loss of critical details.

Q2: A customer says "I AM SO ANGRY RIGHT NOW!!!" about a simple billing charge of \$5. Should the agent escalate to a human?

- A) Yes, high sentiment indicates a complex issue
- B) Yes, capital letters indicate urgency
- C) No, sentiment intensity does not indicate issue complexity – the billing issue is simple and resolvable
- D) No, but reduce the agent's confidence score

Answer: C – Sentiment ≠ complexity. An angry customer with a simple \$5 billing issue doesn't need human escalation. The agent should resolve the simple issue while acknowledging the customer's frustration.

Q3: An extraction system reports 97% overall accuracy. A stakeholder asks if it's ready for production. What additional information is needed?

- A) The system is ready at 97%
- B) Check if accuracy is consistent across all document types and fields

- C) Run more documents through to increase the sample size
- D) Compare against competitor systems

Answer: B – Aggregate accuracy can mask poor performance on specific document types. 97% overall might hide 45% accuracy on handwritten documents. Break down by type and field before declaring production readiness.

Q4: Two research subagents return different values for the same metric. What should the coordinator do?

- A) Average the two values
- B) Use the most recent value
- C) Annotate the conflict with source attribution and let the user decide
- D) Discard both and search again

Answer: C – Conflicting data should be annotated with source attribution rather than silently resolved. This preserves transparency and lets downstream consumers evaluate the discrepancy.

Q5: A long-running agent enables server-side compaction. The integration appends only `response.content[0].text` to its message history each turn. What goes wrong?

- A) Nothing – text is all the API needs
- B) Compaction state is lost silently, because the compaction blocks in `response.content` are what the API uses to replace compacted history
- C) The request fails with a 400 on the first compaction
- D) Compaction triggers too early

Answer: B – Append the whole `response.content`, not the extracted text. Dropping the non-text blocks destroys the compaction state without raising an error, so the failure only surfaces deep into a long conversation.

Q6: An agentic workflow keeps getting truncated mid-task when it hits `max_tokens`. Which mechanism lets the model pace itself instead?

- A) Raise `max_tokens` and hope for the best
- B) A task budget (`output_config.task_budget`), which the model can see and plan around
- C) A lower `effort` setting
- D) An iteration cap in the client loop

Answer: B – `max_tokens` is an enforced ceiling the model is unaware of; a task budget injects a countdown the model sees during generation, so it finishes gracefully. Lower `effort` reduces spend but does not communicate a ceiling, and a client-side iteration cap is the anti-pattern from Domain 1 §1.1.

Q7: A custom harness keeps context small by deleting old tool results from the middle of the `messages` array before each request. It worked on Claude Opus 5. After moving to Claude Fable 5.1, requests start failing with a 400 that mentions thinking blocks. What is the correct fix?

- A) Strip all `thinking` blocks from every request
- B) Keep the history append-only and move the trimming to server-side context editing (`clear_tool_uses`) or compaction, which don't count as edits
- C) Switch `thinking` to `{"type": "disabled"}`
- D) Delete the tool results *and* the thinking blocks that follow them

Answer: B – Fable 5.1's thinking blocks are valid only against the exact prefix that preceded them, so any client-side edit to earlier turns invalidates every later block.

Server-side context editing and compaction are exempt because the check compares the conversation as you sent it. A always works but throws away the reasoning and restarts the prompt cache on every request; C is a 400 on Fable 5.1 and Opus 5.5 (thinking is always on); D still edits the middle of the transcript.

Q8: An application uses on-demand compaction. It puts the returned `compaction` block first in `messages`, but leaves the messages the block summarized in place after it. Requests keep returning 200. What is happening?

- A) The request should have returned `compaction_block_misplaced`; the API is being lenient
- B) Nothing was saved – the API sends the summarized messages to Claude again after the summary, and no error is raised
- C) The API deduplicates the summarized messages against the block automatically
- D) The block is dropped because its signature no longer matches the messages

Answer: B – After the swap, the block stands in for the messages it summarized, and the API passes every message after it to Claude unchanged, so leaving them in place re-sends them with no error. `compaction_block_misplaced` (A) fires only when summarized messages remain *in front of* the block. Leaving the block out of a later request is the other silent mistake: Claude simply gets no summary. Signatures (D) are checked, but only against edits to the block itself.

Claude Certified Associate – Foundations (CCA0-F)

Overview guide for the **Claude Certified Associate – Foundations** exam – the entry-level, non-developer credential in the Claude certification family. Facts below are taken from the official Exam Guide v1.0 (effective July 2026).

Looking for the Architect exam? The full deep-dive study guide in this repo covers [CCAR-F](#). This page is an overview of a sibling certification.

What It Validates

That you can apply Claude to complete business and productivity tasks with minimal guidance: using built-in platform features and tools to streamline workflows, identifying opportunities to improve processes, selecting approaches that balance quality, efficiency, and cost, and recognizing limitations – escalating more complex or technical work to Claude Architects and Developers.

Intended for: professionals who use Claude as a productivity tool – operations, marketing, project management, education, communications, and general knowledge work. Candidates typically have limited to moderate technical expertise, positioned between casual prompt users and technical AI practitioners.

Not intended for: software developers building against APIs or designing agentic systems – that scope belongs to the Developer and Architect credentials.

Exam at a Glance

Exam code	CCA0-F
Items	60 (multiple-choice and multiple-response; each item states how many responses to select)
Time limit	120 minutes
Delivery	Proctored – online proctored and/or Pearson VUE test center
Passing score	720 scaled (scale 100–1,000)
Exam fee	\$99 USD
Validity	12 months from award
Result reporting	Pass/fail with scaled score, plus percent-correct by domain
Prerequisites	None mandatory – recommended experience only

Domain Blueprint

#	Domain	Weight
1	Prompting and Task Execution	14%
2	Output Evaluation and Validation	21%
3	Product and Model Selection	12%
4	Workflow Integration and Solution Design	16%
5	Configuration and Knowledge Management	12%
6	Governance, Risk, and Responsible Use	15%
7	Troubleshooting and Optimization	10%

Note the emphasis: output evaluation (21%), workflow design (16%), and governance (15%) together make up more than half the exam – this credential tests **judgment about using AI responsibly**, more than prompting technique.

Key objectives per domain

1. **Prompting and Task Execution** – effective prompts for business tasks, task decomposition, iterating prompts, adapting strategy by task type (analysis, research, drafting, brainstorming).
2. **Output Evaluation and Validation** – evaluating accuracy and completeness, spotting hallucinations/inconsistencies/bias, fact-checking, knowing when human review is required, adapting outputs for an audience, choosing output formats (artifacts, inline, structured data).
3. **Product and Model Selection** – choosing product features (Projects, research mode, chat, artifacts), differentiating model tiers (Haiku, Sonnet, Opus), aligning model choice with cost/speed/quality, managing context limits and memory (when to restart, summarize, or persist).
4. **Workflow Integration and Solution Design** – applying Claude to requirements analysis, research, planning, process optimization; integrating Claude into existing workflows; communicating Claude's value and limitations to stakeholders.
5. **Configuration and Knowledge Management** – configuring Claude Projects with instructions and knowledge sources, managing connectors (e.g. Google Drive, Gmail), writing effective system-level instructions, maintaining configurations over time.
6. **Governance, Risk, and Responsible Use** – appropriate vs inappropriate use cases, data sensitivity/regulatory/privacy considerations, organizational AI policy compliance, ethical implications.
7. **Troubleshooting and Optimization** – diagnosing underperforming prompts and poor outputs, adjusting based on feedback, optimizing workflows.

How to Prepare (per the official guide)

- Study the blueprint and self-assess against each objective
- Review official documentation for Projects, Artifacts, Memory, Skills, and Code Execution
- Practice structuring prompts, decomposing tasks, and iterating on outputs
- Build a real workflow: configure a Project with instructions and knowledge sources, then evaluate outputs for accuracy and bias
- Practice responsible-use judgment: data sensitivity, appropriate use cases, when to escalate or seek human review

Much of this repo's [Domain 4 \(explicit criteria, few-shot\)](#) and [Domain 5 \(context management, escalation, provenance\)](#) material transfers directly – at a less technical depth.

Registration and Policies

1. Register via the exam's page on the **Anthropic Partner Academy** (partner-tier discounts apply at checkout), then schedule through **Pearson VUE** – online proctoring or a test center.
 2. Cancel/reschedule up to 24 hours before the appointment; changes within 24 hours forfeit the fee.
 3. Retakes: up to 4 attempts per rolling 12 months, with waiting periods after each failed attempt (14 / 30 / 90 days), per Pearson VUE program policy.
 4. Government-issued photo ID required; the name must exactly match your registration.
 5. Renewal: the credential is valid 12 months; renewing on time is free – review what changed since you certified and pass a non-proctored assessment. If it lapses, the full exam fee applies again. Registration requires a partner-domain email address. Delivery moved to Pearson VUE and digital badging to Credly on 2026-06-30; candidates must be at least 18. Partner-tier discounts apply at checkout – 50% for Select, Preferred, and Global Premier partners, 100% for Global Premier partners through 2026-12-31.
-

Resources

- [Official CCA0-F Exam Guide \(PDF, v1.0\)](#)
- [Pearson VUE – Anthropic certification program](#)
- [Anthropic Skilljar courses](#)

Claude Certified Architect – Professional (CCAR-P)

Overview guide for the **Claude Certified Architect – Professional** exam – the advanced tier above the Architect Foundations exam this repo covers in depth. Facts below are taken from the official Exam Guide v1.0 (effective July 2026).

Preparing for Foundations first? Start with the full [CCAR-F study guide](#) – Anthropic's Professional blueprint assumes you operate comfortably at that level.

What It Validates

That you can design, build, and deliver production-grade AI solutions on the Claude platform as an architect who owns the full lifecycle: selecting models, architectures, and API patterns; prompt and context engineering; integrating Claude into enterprise systems; and incorporating evaluation, security, compliance, and governance into designs – plus the distinctly Professional-level skill of **stakeholder communication and lifecycle management**.

Intended for: mid- to senior-level solution architects, AI/ML engineers, and technical leads. Recommended (not required): 3+ years in systems architecture or platform engineering, 6+ months hands-on with Claude (or comparable LLM systems) in production, experience delivering end-to-end systems from discovery through operationalization.

Not intended for: entry-level developers, casual users, or roles limited to isolated tasks (e.g. prompt writing) without broader system-design responsibility.

Exam at a Glance

Exam code	CCAR-P
Items	63 (multiple-choice and multiple-response; each item states how many responses to select)
Time limit	120 minutes
Delivery	Proctored – online proctored and/or Pearson VUE test center
Passing score	720 scaled (scale 100–1,000)
Exam fee	\$175 USD
Validity	12 months from award
Result reporting	Pass/fail with scaled score, plus percent-correct by domain
Prerequisites	None mandatory – recommended experience only

Domain Blueprint

Seven domains vs Foundations' five. The Foundations themes reappear at higher altitude, and two areas are effectively new: RAG-heavy Integration, and Stakeholder Communication & Lifecycle Management.

#	Domain	Weight
1	Solution Design & Architecture	17%
2	Claude Models, Prompting & Context Engineering	13%
3	Integration	19%
4	Evaluation, Testing & Optimization	16%
5	Governance, Safety & Risk Management	14%
6	Stakeholder Communication & Lifecycle Management	14%
7	Developer Productivity & Operational Enablement	7%

Key objectives per domain

- 1. Solution Design & Architecture (17%)** – translating business problems into Claude-based solutions; end-to-end architectures (input → processing → output → feedback loops); choosing among workflow, agentic, and augmented-LLM patterns; multi-agent systems and orchestration; decomposition; aligning to business value pillars (efficiency, cost, performance SLAs).
- 2. Claude Models, Prompting & Context Engineering (13%)** – model selection by trade-off; system prompts, templates, guardrails; zero-shot/few-shot/chain-of-thought; context-window and token optimization; prompt reuse (caching, modular prompts, Skills).
- 3. Integration (19%)** – the heaviest domain: tool/agent capability-bloat evaluation, authn/authz gap analysis, accuracy-latency trade-offs, observability at scale, **RAG pipeline design** (chunking, indexing, retrieval matched to data shape and query pattern), choosing integration mechanisms (MCP vs API/CLI vs agent-to-agent), progressive-discovery vs monolithic context strategy.
- 4. Evaluation, Testing & Optimization (16%)** – evaluation metrics (accuracy, latency, cost, safety, security), eval datasets and test frameworks, A/B testing, diagnosing prompt failure/hallucination/model mismatch, token/latency/cost optimization, logging and observability.
- 5. Governance, Safety & Risk Management (14%)** – guardrails and safety controls, LLM failure modes, human-in-the-loop validation, regulatory compliance (GDPR, HIPAA, FedRAMP), ethical AI (bias, fairness, transparency).
- 6. Stakeholder Communication & Lifecycle Management (14%)** – structured discovery and requirements gathering, communicating architectural decisions and trade-offs, feedback loops and expectation/SLA alignment, architecture documentation, supporting lifecycle phases (discovery, design, handoff, monitoring, iteration). This domain does not exist on CCAR-F.
- 7. Developer Productivity & Operational Enablement (7%)** – configuring Claude tooling for teams (e.g. Claude Code), improving developer workflows with AI-assisted tooling, supporting debugging and operational issues.

Sample-question flavor (from the official guide)

The published samples test least-privilege tool configuration (remove unneeded refund/delete tools rather than log or confirm), prompt-caching architecture (stable prefix first + caching to cut latency *and* cost), and RAG debugging (confident-but-wrong answers after a document refresh → suspect retrieval/indexing first). Expect judgment-under-tradeoffs questions, not recall.

Relationship to CCAR-F and This Repo

- All five CCAR-F domains reappear inside CCAR-P domains 1–5 – the [existing deep-dives](#) remain the right base layer.
- Study the deltas on top: **RAG architecture** (chunking/indexing/retrieval strategy), **evaluation frameworks and A/B testing**, **regulated-industry compliance**, and **stakeholder communication / lifecycle management** – none of which the Foundations guide covers today.
- Anthropic recommends sitting CCAR-F first unless you already have deep, current production experience.

How to Prepare (per the official guide)

- Study the blueprint and self-assess against each objective
- Review official documentation for the Claude API, models, prompt engineering, MCP, and Skills
- Build and operate at least one end-to-end Claude solution including RAG, evaluation, and observability
- Practice architectural decision-making: model selection, integration protocols, security trade-offs

Registration and Policies

1. Register via the exam's page on the **Anthropic Partner Academy** (partner-tier discounts apply at checkout), then schedule through **Pearson VUE** – online proctoring or a test center.
2. Cancel/reschedule up to 24 hours before the appointment; changes within 24 hours forfeit the fee.
3. Retakes: up to 4 attempts per rolling 12 months, with waiting periods after each failed attempt (14 / 30 / 90 days), per Pearson VUE program policy.
4. Renewal: the credential is valid 12 months; renewing on time is free – review what changed since you certified and pass a non-proctored assessment. If it lapses, the full exam fee applies again. Registration requires a partner-domain email address. Delivery moved to Pearson VUE and digital badging to Credly on 2026-06-30; candidates must be at least 18. Partner-tier discounts apply at checkout – 50% for Select, Preferred, and Global Premier partners, 100% for Global Premier partners through 2026-12-31.

Resources

- [Official CCAR-P Exam Guide \(PDF, v1.0\)](#)
- [Pearson VUE – Anthropic certification program](#)
- [CCAR-F study guide in this repo](#) – the base layer

Claude Certified Developer – Foundations (CCDV-F)

Overview guide for the **Claude Certified Developer – Foundations** exam – the hands-on implementation credential in the Claude certification family. Facts below are taken from the official Exam Guide v1.0 (effective July 2026).

Looking for the Architect exam? The full deep-dive study guide in this repo covers [CCAR-F](#). This page is an overview of a sibling certification – with heavy content overlap, mapped below.

What It Validates

That you can build, integrate, and ship production-grade applications, agents, and workflows on the Claude platform at a foundational level: building agents with the Claude Agent SDK and custom agent loops, integrating Claude through the API and client SDKs (streaming, error handling, multi-format input), operating Claude Code (Skills, CLAUDE.md, settings.json, plugins), prompt and context engineering, designing and running evals, model/cost/latency optimization, secure-by-design practices and hook-based guardrails, and building custom tools and MCP servers.

Intended for: AI/ML engineers, technical leads, and senior software engineers. Recommended (not required): 1–5 years of software engineering, 6+ months hands-on with Claude or comparable LLM systems, Python and/or TypeScript, REST APIs and CLI fluency.

Not intended for: non-technical users, or roles limited to prompt writing without broader application-development responsibility.

Exam at a Glance

Exam code	CCDV-F
Items	53 (multiple-choice and multiple-response; each item states how many responses to select)
Time limit	120 minutes
Delivery	Proctored – online proctored and/or Pearson VUE test center
Passing score	720 scaled (scale 100–1,000)
Exam fee	\$125 USD
Validity	12 months from award
Result reporting	Pass/fail with scaled score, plus percent-correct by domain
Prerequisites	None mandatory – recommended experience only

Domain Blueprint

Unlike the Architect Foundations exam (5 evenly weighted domains), CCDV-F has 8 domains with very uneven weights – a third of the exam is Applications and Integration:

#	Domain	Weight
1	Agents and Workflows	14.7%
2	Applications and Integration	33.1%
3	Claude Code	3.1%
4	Eval, Testing, and Debugging	2.6%
5	Model Selection and Optimization	16.8%
6	Prompt and Context Engineering	11.0%
7	Security and Safety	8.1%
8	Tools and MCPs	10.6%

What each domain covers

1. **Agents and Workflows (14.7%)** – workflow-vs-agent decision criteria, manager/supervisor hierarchies, subagents; building with the Claude Agent SDK, custom loops and harnesses, hosted vs self-hosted deployment, hooks for deterministic actions; agent design patterns (tool-use loops, sub-agents, memory, context-window management) and frameworks.
2. **Applications and Integration (33.1%)** – the big one. Requirements and systems-lifecycle basics; Claude API mechanics (messages, tools, streaming, vision, thinking, caching, third-party vendors, batch vs realtime); software-engineering foundations (REST, JSON, async, version control, refactoring); application design (how Claude interprets instructions across Claude Code / Desktop / claude.ai / API / SDKs, content boundaries, schema design, session hygiene); configuration management (CLAUDE.md, settings.json, model pinning, prompt versioning).
3. **Claude Code (3.1%)** – core components (Rules, Skills, Commands, Agents, Agent Memory), session management, slash commands, headless mode, streaming mode, auto-mode, the CLAUDE.md hierarchy, settings.json.
4. **Eval, Testing, and Debugging (2.6%)** – error-type identification, recovery strategies, trace analysis, isolating integration-layer vs model-output problems.
5. **Model Selection and Optimization (16.8%)** – LLM fundamentals (tokens, context windows, sampling, non-determinism), model options (fast mode, extended/adaptive thinking, effort levels), model-tier tradeoffs and breaking changes across releases, token budgeting, prompt caching for cost.
6. **Prompt and Context Engineering (11.0%)** – context-drift/bloat prevention (tool-output pruning, compaction), context isolation via subagents, prompt engineering principles, structured-output patterns, defensive parsing, skepticism toward confident output.
7. **Security and Safety (8.1%)** – prompt-injection awareness and mitigation, jailbreak defense, untrusted input, PII, guardrail layering, hooks as safety controls, secrets and key management.
8. **Tools and MCPs (10.6%)** – tool use and function calling, tool-description writing, client-side vs server-side tools, approval patterns, MCP server authoring and deployment, stdio vs socket transports.

Overlap with This Repo's CCAR-F Material

A large share of CCDV-F content is already covered in the Architect guide, at architect altitude:

CCDV-F domain	Covered in
Agents and Workflows	d1 – Agentic Architecture
Tools and MCPs	d2 – Tool Design & MCP
Claude Code	d3 – Claude Code Configuration
Prompt and Context Engineering	d4 – Prompt Engineering + d5 – Context Management
Applications and Integration / Model Selection	Partially in d1/d4; the API-mechanics and cost-optimization depth is CCDV-F-specific

The developer exam goes deeper on API mechanics, SDK usage, evals, security, and cost optimization than the Architect Foundations blueprint does.

How to Prepare (per the official guide)

- Study the blueprint and self-assess against each skill
- Build and ship at least one real Claude application end-to-end: API integration, an agent loop or Agent SDK build, custom tools or an MCP server, and an eval
- Review official documentation for the Claude API, Agent SDK, Claude Code, and MCP
- Practice cost levers hands-on: model-tier selection, prompt caching, batch processing

Registration and Policies

1. Register via the exam's page on the **Anthropic Partner Academy** (partner-tier discounts apply at checkout), then schedule through **Pearson VUE** – online proctoring or a test center.
2. Cancel/reschedule up to 24 hours before the appointment; changes within 24 hours forfeit the fee.
3. Retakes: up to 4 attempts per rolling 12 months, with waiting periods after each failed attempt (14 / 30 / 90 days), per Pearson VUE program policy.
4. Renewal: the credential is valid 12 months; renewing on time is free – review what changed since you certified and pass a non-proctored assessment. If it lapses, the full exam fee applies again. Registration requires a partner-domain email address. Delivery moved to Pearson VUE and digital badging to Credly on 2026-06-30; candidates must be at least 18. Partner-tier discounts apply at checkout – 50% for Select, Preferred, and Global Premier partners, 100% for Global Premier partners through 2026-12-31.

Resources

- [Official CCDV-F Exam Guide \(PDF, v1.0\)](#)
- [Pearson VUE – Anthropic certification program](#)
- [Claude API docs](#) · [Claude Code docs](#) · [Agent SDK](#) · [MCP](#)